TECHNICAL DOCUMENTATION

Mini Security Operations Center (SIEM)

A monitored Spring Boot application, a log pipeline into OpenSearch, a rule-based detection engine, an incident tracker and a React SOC dashboard, all run with Docker Compose.

Java 21 | Spring Boot 3.3 | Spring Security + JWT | OpenSearch 2.17 | Fluent Bit 3.1 | PostgreSQL 16 | React 18 + Vite | nginx | Docker Compose | Python simulators

Repository	github.com/SaiPrakashBV/Mini-SOC-Project
Author	Sai Prakash
Covers	architecture, every source file, data model, deployment, security controls, attack simulation, testing and troubleshooting
Document date	November 2026

Contents

1	Project overview & outcomes	what it is, what you get
2	Users, roles & accounts	who uses what
3	System architecture	components, stack, trust boundaries
4	End-to-end flow	from attack to resolved incident
5	Repository & file reference	every file and its purpose
6	Backend 1 - target-app	JWT, RBAC, WAF, audit logging
7	Log pipeline - Fluent Bit & OpenSearch	collection, indexing, retention
8	Backend 2 - detection-service	rule engine and all 16 rules
9	Backend 3 - incident-tracker	REST API, workflow, de-duplication
10	Data stores	PostgreSQL schema, OpenSearch indices
11	Frontend - SOC dashboard	React, routing, API client, nginx
12	Service communication	ports, protocols, authentication
13	Docker deployment & operations	compose, images, start/stop/reset
14	Security measures & safeguards	what protects what, and where
15	Attack simulation & expected outcomes	six simulators and scripts
16	Testing	automated test suites
17	Troubleshooting guide	symptom, cause, fix
18	Extending & hardening	new rules, production checklist
A	Appendix - API & configuration reference	endpoints, env vars, events

1 Project overview & outcomes

Mini SOC is a self-contained lab that reproduces the core loop of a real Security Operations Center: **an application produces security telemetry, a SIEM collects and analyses it, detections raise alerts, and analysts investigate and close incidents.**

What the system does

1. **Runs a realistic target.** `target-app` is a small e-commerce style API with login, users, orders and admin endpoints. It is protected by JWT authentication, role-based access control, per-object ownership checks and a minimal web application firewall (WAF).
2. **Emits structured security events.** Every login, denied request, invalid token, blocked payload and HTTP request is written as one JSON line. Each line carries the timestamp, source IP, username, token ID and a correlation ID.
3. **Collects and indexes them.** Fluent Bit tails the log file and ships it to OpenSearch, which stores it in daily `security-events-*` indices.
4. **Detects attacks.** `detection-service` runs 16 YAML-defined rules every 15 seconds as OpenSearch aggregation queries. The rules cover brute force, credential stuffing, account takeover, session hijacking, injection, Log4Shell, reconnaissance, scanners, scraping, privilege escalation and DoS.
5. **Manages incidents.** Fired alerts become incidents in `incident-tracker` (PostgreSQL). There, analysts assign them, change their status, add notes and record resolutions, with a full audit trail.
6. **Visualises it.** The React dashboard shows KPIs, alerts by severity, an activity timeline, affected users, top attacker IPs, the incident workflow and an event explorer.
7. **Simulates attackers.** Six standalone simulators and two all-in-one scripts generate realistic attack traffic, so every detection can be demonstrated on demand.

Outcome at a glance

Area	Result
Services	8 containers (+2 optional): target-app, fluent-bit, opensearch, opensearch-init, detection-service, incident-tracker, postgres, frontend; attacker and opensearch-dashboards on demand
Detections	16 rules: 9 threshold rules, 3 cardinality rules (distinct IPs / paths), 1 correlation rule (failures followed by a success) and 3 rules on WAF payload signatures
Event types	10: LOGIN_SUCCESS, LOGIN_FAILURE, TOKEN_INVALID, UNAUTHORIZED, ACCESS_DENIED, ADMIN_ACTION, ORDER_CREATED, RESOURCE_OWNERSHIP_VIOLATION, MALICIOUS_REQUEST, HTTP_REQUEST
Incident API	8 incident endpoints (the 7 required: create, list/filter, detail, status, assign, notes, stats, plus severity) and 2 analyst-directory endpoints
Verification	24 automated tests (JUnit/MockMvc) plus an end-to-end run of all simulators against the full Docker stack: every expected rule fired
Time to detect	about 5 s for log shipping + up to 15 s poll interval = typically under 20 s from attack to incident

> **Lab, not production**
> The project is designed to be run locally to learn and demonstrate SOC concepts. Some infrastructure security is deliberately relaxed for convenience (for example, the OpenSearch security plugin is off and only bound to localhost). Chapter 14 lists every safeguard that *is* in place, and chapter 18 lists what to change before exposing anything.

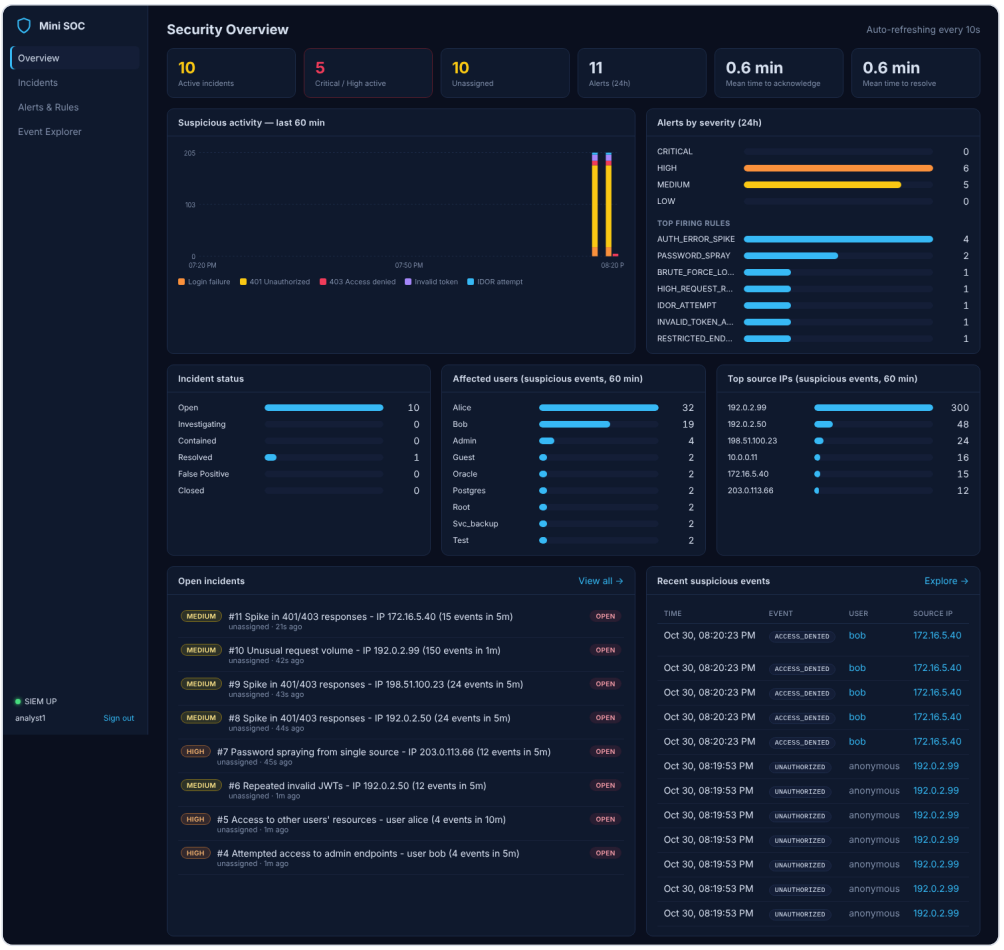


Figure 1 - The SOC dashboard overview after running the attack scenarios.

2 Users, roles & accounts

There are four kinds of actor. Passwords are never stored in the repository. They are generated into `.env` by `scripts/generate-env.sh` / `.ps1` and injected into the containers as environment variables.

2.1 Application users (target-app)

Username	Roles	Password variable	Can do
alice	USER	`DEMO_ALICE_PASSWORD`	Log in, view her own profile and orders (1001, 1002), create orders
bob	USER	`DEMO_BOB_PASSWORD`	Same as alice, for his orders (1003, 1004)
admin	USER, ADMIN	`DEMO_ADMIN_PASSWORD`	Everything above plus all orders, GET /api/users and /api/admin/*

Users are loaded at start-up from `app.security.users` into Spring Security's `InMemoryUserDetailsManager`, with BCrypt-hashed passwords. Roles become JWT claims (`ROLE_USER` , `ROLE_ADMIN`).

2.2 SOC analysts (dashboard, incident-tracker, detection-service)

Username	Roles	Password variable	Purpose
analyst1	ANALYST	`ANALYST1_PASSWORD`	Tier-1 analyst: triage, assign, investigate, resolve
lead	ANALYST, LEAD	`SOC_LEAD_PASSWORD`	SOC lead. Today LEAD grants nothing extra; it is a hook for future approval rules

The same two accounts are configured in both backend APIs, so a single dashboard login works for both. The browser sends them as HTTP Basic credentials through the nginx proxy.

2.3 Machine principals

Principal	How it authenticates	Permissions
detection-service	`X-API-Key: $TRACKER_API_KEY` to incident-tracker	ROLE_SERVICE: may only POST /api/incidents and GET; cannot change status, assign or add notes
detection-service	optional Basic auth to OpenSearch (`OPENSEARCH_USER`)	read security-events-*, write soc-alerts
incident-tracker	`POSTGRES_USER` / `POSTGRES_PASSWORD`	owns the soc database schema (Flyway)
soc_readonly (DB role)	NOLOGIN by default; enable with ALTER ROLE ... LOGIN PASSWORD	SELECT on all tables, for reporting/BI

2.4 Attackers (simulated)

The simulators impersonate many attackers by setting `X-Forwarded-For`, which target-app trusts because `TRUST_FORWARDED_FOR=true` in the lab. This is how one machine produces botnets, Tor exit nodes and scanners with distinct source IPs. See chapter 15.

3 System architecture

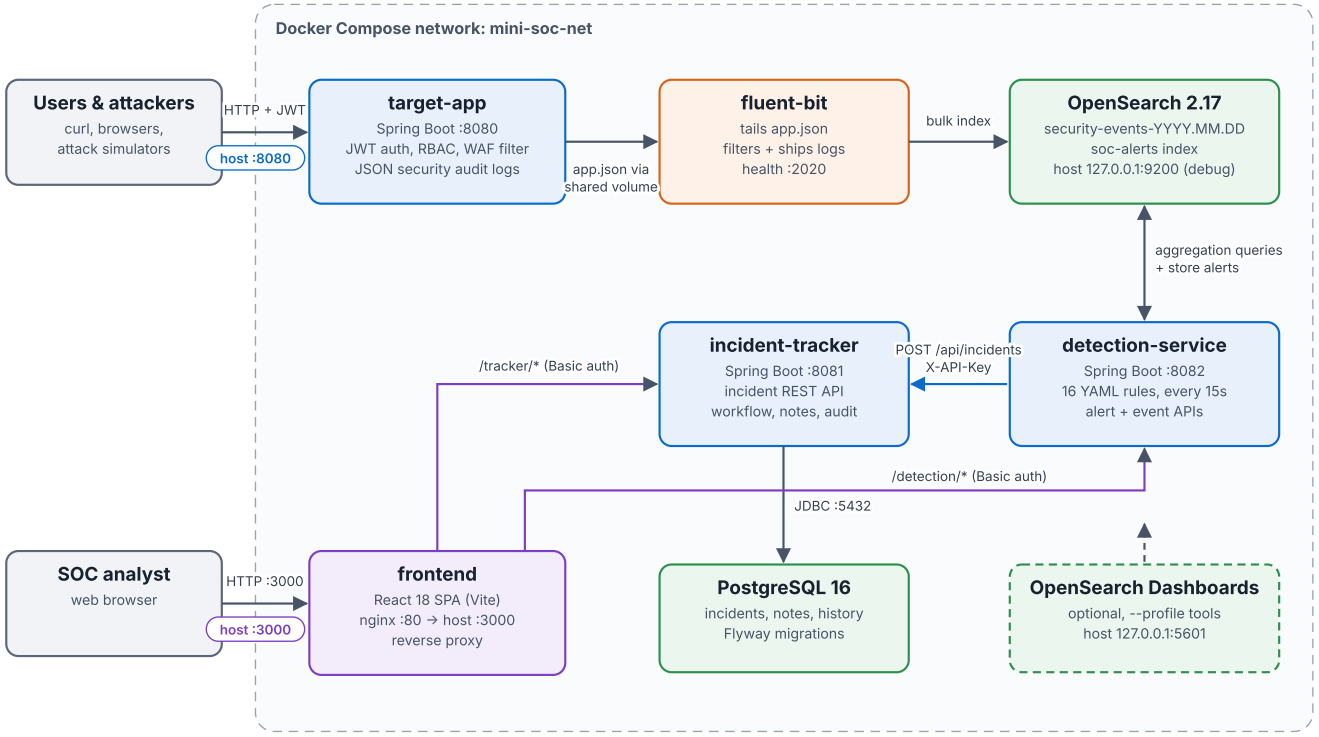


Figure 2 - Containers, data flows and published ports. Purple arrows are dashboard API calls, blue is service-to-service.

3.1 Components and responsibilities

Component	Technology	Responsibility	Port
target-app	Java 21, Spring Boot 3.3.5, Spring Security, jjwt	The monitored application; produces security events	8080 (published)
fluent-bit	Fluent Bit 3.1.9	Tails app.json, filters, ships to OpenSearch	2020 (internal metrics)
opensearch	OpenSearch 2.17.1 single node	SIEM storage and search: events + alerts	9200 (127.0.0.1 only)
opensearch-init	curl 8.10.1 + init.sh	One-shot: templates, alert index, ISM policy, sample monitor	-
detection-service	Spring Boot, RestClient	Runs rules, stores alerts, creates incidents, serves event/alert APIs	8082 (internal)
incident-tracker	Spring Boot, JPA/Hibernate, Flyway	Incident lifecycle, notes, history, statistics	8081 (internal)
postgres	PostgreSQL 16 (alpine)	Durable incident storage	5432 (internal)
frontend	React 18, Vite 5, nginx 1.27	SOC dashboard + reverse proxy to the two APIs	3000 → 80 (published)
opensearch-dashboards	OpenSearch Dashboards 2.17.1	Optional raw-event exploration (--profile tools)	5601 (127.0.0.1)
attacker	python:3.12-alpine	Runs attack-simulators/*.py inside the network on demand	-

3.2 Why this shape

- **Producer and SIEM are decoupled by a file.** target-app only writes JSON lines to a shared volume. If OpenSearch or the detection engine is down, the application keeps working, and Fluent Bit catches up from its saved file offsets (`tail.db`) when it comes back.
- **Detection is pull-based.** The engine queries aggregations over a sliding window instead of streaming every event. This is simple, stateless across restarts (except suppression), and works the same way real SIEM scheduled searches do.
- **Incidents live in a relational DB.** Workflow state, assignments and audit history need transactions, constraints and optimistic locking. OpenSearch is used for what it is good at: search and aggregation.
- **One public entry point for analysts.** The two backend APIs publish no host ports. The browser reaches them only through nginx on the same origin, which avoids CORS and keeps the attack surface small.

3.3 Trust boundaries

Client → target-app	JWT (HS256, 30 min, issuer checked)
Fluent Bit → OpenSearch	internal network only; security plugin off (lab)
detection → OpenSearch	internal network; optional Basic auth via env
detection → incident-tracker	X-API-Key, constant-time compare, ROLE_SERVICE
browser → nginx → backends	HTTP Basic analyst accounts, ROLE_ANALYST
incident-tracker → PostgreSQL	DB user/password from .env, no host port

Figure 3 - How each hop authenticates.

4 End-to-end flow

The specification's reference scenario is "five invalid logins in two minutes". This chapter follows those requests through every component.

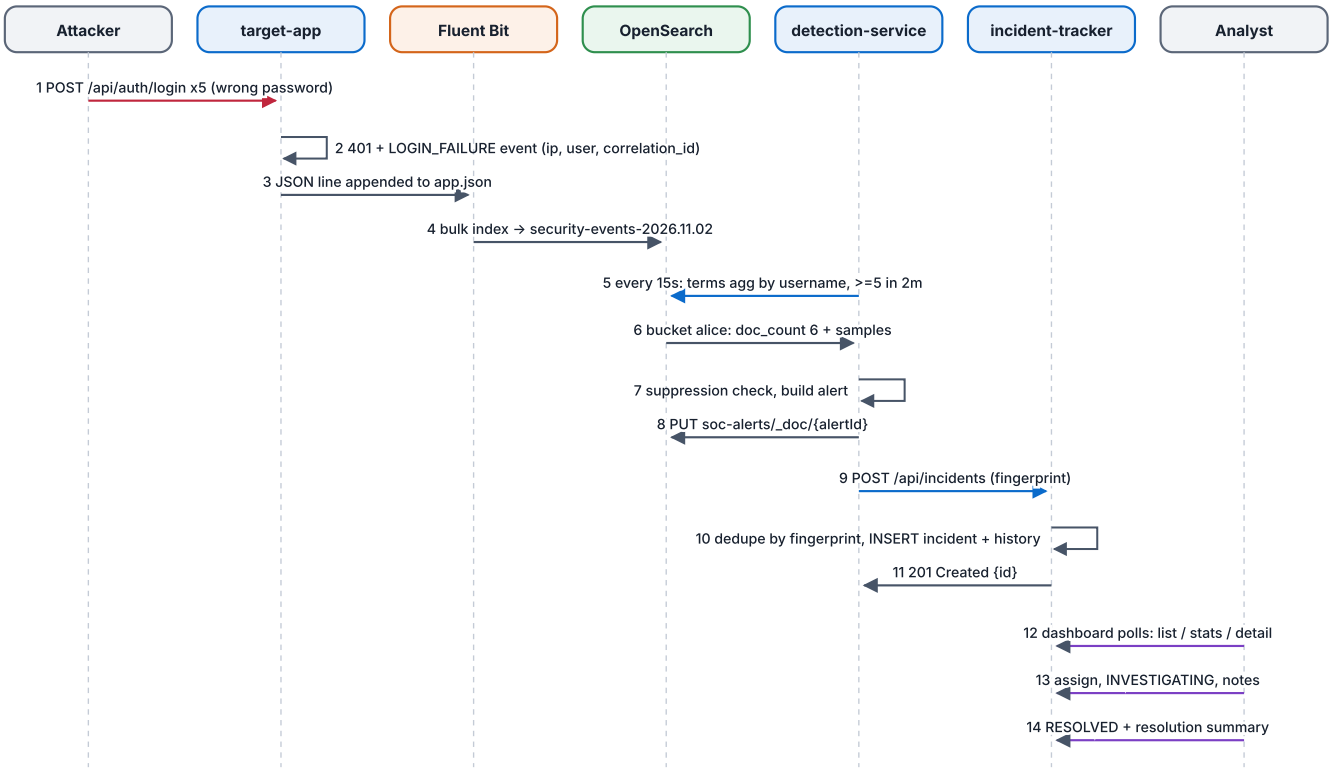


Figure 4 - Sequence from brute-force attempts to a resolved incident.

4.1 Step by step

1. **Simulated attack.** `./scripts/simulate-attacks.sh bruteforce` (or simulator 1) posts wrong passwords for `alice` with `X-Forwarded-For: 203.0.113.66`.
2. **Security event.** `AuthController` catches the `AuthenticationException` and calls `SecurityEventLogger.failure(LOGIN_FAILURE, ...)`. The request filter adds the correlation ID and source IP. A second event, `HTTP_REQUEST` with status 401, is written when the request finishes.
3. **Collection.** Fluent Bit reads the new line within about 2 s, parses the JSON and keeps it, because `log_type=security_audit`. It then bulk-indexes it into `security-events-2026.11.02`.
4. **Detection.** Every 15 s the engine sends the `BRUTE_FORCE_LOGIN` query: LOGIN_FAILURE events in the last 2 minutes, grouped by username, keeping only buckets with at least 5 events.
5. **Alert.** The `alice` bucket qualifies. The suppression cache has no entry for `BRUTE_FORCE_LOGIN:username:alice`, so an alert is stored in `soc-alerts`.
6. **Incident.** The alert is POSTed to the tracker with the fingerprint. No active incident has that fingerprint, so a new HIGH incident is inserted, together with a CREATED history row.
7. **Investigation.** An analyst opens the incident in the dashboard and sees the evidence samples and alice's event timeline. They assign it, move it to INVESTIGATING, add notes, and resolve it with a summary.

Event written by target-app (illustrative)


```
{
  "@timestamp": "2026-11-02T10:15:42.318Z",
  "message": "Authentication failed",
  "logger_name": "SECURITY_AUDIT",
  "level": "WARN",
  "correlation_id": "8f0c2c1e-5b1d-4d55-9a57-2f3b9a0a6c11",
  "source_ip": "203.0.113.66",
  "event_type": "LOGIN_FAILURE",
  "event_category": "authentication",
  "outcome": "FAILURE",
  "log_type": "security_audit",
  "username": "alice",
  "reason": "BadCredentialsException",
  "service": "target-app"
}
```


Incident request sent by detection-service


```
{
  "title": "Potential brute-force login - user alice (6 events in
2m)",
  "severity": "HIGH",
  "ruleId": "BRUTE_FORCE_LOGIN",
  "fingerprint": "BRUTE_FORCE_LOGIN:username:alice",
  "affectedUser": "alice",
  "sourceIp": "203.0.113.66",
  "detectedAt": "2026-11-02T10:15:40.112Z",
  "evidence": { "count": 6, "threshold": 5, "window": "2m",
               "sourceIps": ["203.0.113.66"], "samples": [ ... 5
newest events ... ] }
}
```


4.2 What happens if the attack continues

For the next 2 minutes, which is the rule window and the default suppression period, the same user does not alert again. After that, a new alert with the same fingerprint is folded into the **existing** open incident. Its `detectionCount` goes up, a RETRIGGERED history entry and a note are added, and the severity is raised if the new alert is more severe (for example, CRITICAL once 20 failures are reached). Once the incident is RESOLVED, CLOSED or FALSE_POSITIVE, a new detection opens a fresh incident.

5 Repository & file reference

Every tracked file in the repository, with what it is for. In Java paths, `…/` stands for `src/main/java/com/minisoc/` and `…test/` for `src/test/java/com/minisoc/`.

Top-level layout

```
mini-soc/
├── target-app/          Spring Boot target application (attack surface + event source)
├── incident-tracker/    Spring Boot incident REST API (PostgreSQL)
├── detection-service/   Spring Boot detection engine (OpenSearch queries)
├── frontend/            React SOC dashboard served by nginx
├── fluent-bit/          log forwarding config
├── opensearch/          index templates, retention policy, sample monitor, init script
├── database/            PostgreSQL first-run initialisation
├── attack-simulators/   six standalone Python attack campaigns
├── scripts/             secret generation + all-in-one attack scripts (bash & PowerShell)
├── docker-compose.yml
└── .env.example         -> copied to .env (git-ignored) with real secrets
```

Repository root

File	Purpose
`.env.example`	Template of every secret and tuning variable. Copied to the git-ignored .env by the generate-env scripts.
`.gitattributes`	Forces LF line endings for shell scripts, configs, SQL, YAML, JSON and Python so they work inside Linux containers even when cloned on Windows.
`.gitignore`	Keeps secrets (.env), Maven target/ folders, node_modules, dist and Python caches out of git.
`README.md`	Quick start, architecture sketch, rule table, API summary and hardening notes.
`docker-compose.yml`	Defines all services, the mini-soc-net network, named volumes, published ports, health checks and start-up order.

target-app: the monitored Spring Boot application

File	Purpose
`target-app/Dockerfile`	Multi-stage build: Maven + JDK 21 builds app.jar, a slim JRE 21 image runs it as a non-root user.
`target-app/.dockerignore`	Excludes target/ and logs/ from the Docker build context.
`target-app/pom.xml`	Spring Boot 3.3.5 with web, security, validation, actuator, jjwt 0.12.6 and logstash-logback-encoder.
`target-app/…/target/TargetApplication.java`	Spring Boot entry point; enables @ConfigurationProperties scanning.
`target-app/…/target/config/AppSecurityProperties.java`	Typed, validated app.security.* settings: JWT secret (min 32 chars), TTL, issuer, X-Forwarded-For trust, demo users.
`target-app/…/target/config/SecurityConfig.java`	Stateless Spring Security filter chain: public login, ADMIN-only /api/admin/** and GET /api/users, everything else authenticated; BCrypt in-memory users.
`target-app/…/target/logging/ClientIpResolver.java`	Resolves the client IP; honours X-Forwarded-For only when explicitly trusted.
`target-app/…/target/logging/RequestContext.java`	Shared names for the correlation header, MDC keys (correlation_id, source_ip, username, token_id) and request attributes.
`target-app/…/target/logging/RequestLoggingFilter.java`	Outermost filter: assigns/propagates X-Correlation-ID, records source IP, emits one HTTP_REQUEST event per request, clears the MDC.
`target-app/…/target/logging/SecurityEventLogger.java`	Writes structured audit events (event_type, event_category, outcome, details) to the SECURITY_AUDIT logger.
`target-app/…/target/logging/SecurityEventType.java`	Catalogue of event types with their category - the contract the detection rules depend on.
`target-app/…/target/model/CreateOrderRequest.java`	Validated request body for creating an order.
`target-app/…/target/model/LoginRequest.java`	Validated login body (username/password with length limits).
`target-app/…/target/model/LoginResponse.java`	Login response: access token, type, expiry, username, roles.
`target-app/…/target/model/Order.java`	Order record (id, owner, item, quantity, amount, createdAt).

File	Purpose
target-app/…/target/model/UserProfile.java	Public user profile returned by /api/users.
target-app/…/target/security/AttackSignatureFilter.java	Minimal WAF: regex signatures for JNDI, SQLi, XSS, command injection and path traversal in path/params/headers; blocks with 400 and logs MALICIOUS_REQUEST. Flags scanner user agents.
target-app/…/target/security/AuditAccessDeniedHandler.java	403 handler that logs ACCESS_DENIED with path, user and roles.
target-app/…/target/security/AuditAuthenticationEntryPoint.java	401 handler that logs UNAUTHORIZED for unauthenticated access.
target-app/…/target/security/ErrorResponseWriter.java	Writes consistent JSON error bodies including the correlation id.
target-app/…/target/security/JwtAuthenticationFilter.java	Validates Bearer JWTs, sets the security context and MDC (username, token_id); logs TOKEN_INVALID for bad/expired/unsigned tokens.
target-app/…/target/security/JwtService.java	Issues and verifies HS256 JWTs (subject, roles, jti, issuer, expiry) with jjwt.
target-app/…/target/service/OrderService.java	In-memory order store seeded with four orders (1001-1002 alice, 1003-1004 bob).
target-app/…/target/service/UserDirectory.java	Read-only directory of the configured demo users.
target-app/…/target/web/AdminController.java	/api/admin/stats and /api/admin/maintenance; logs ADMIN_ACTION.
target-app/…/target/web/AuthController.java	POST /api/auth/login: authenticates, issues JWT, logs LOGIN_SUCCESS / LOGIN_FAILURE.
target-app/…/target/web/GlobalExceptionHandler.java	RFC 7807 problem responses for 404 / validation / malformed JSON; leaves AccessDenied to Spring Security.
target-app/…/target/web/NotFoundException.java	Exception mapped to HTTP 404.
target-app/…/target/web/OrderController.java	List/get/create orders; ownership check logs RESOURCE_OWNERSHIP_VIOLATION (IDOR) then 403.
target-app/…/target/web/UserController.java	GET /api/users (admin) and /api/users/me.
target-app/src/main/resources/application.yml	Port, error-detail suppression, actuator exposure, JWT and demo-user settings read from environment variables.
target-app/src/main/resources/logback-spring.xml	JSON logging (LogstashEncoder, UTC) to stdout and a rolling app.json file that Fluent Bit tails.
target-app/…test/target/SecurityFlowTest.java	MockMvc tests: 401 on bad password / missing token, 403 for non-admins, IDOR blocked, tampered token rejected.
target-app/…test/target/AttackDetectionTest.java	Tests for the WAF filter (SQLi, XSS, traversal, command injection, Log4Shell, scanner UA, benign input) and alg=none rejection.
target-app/src/test/resources/application-test.yml	Test-only JWT secret and demo users.

incident-tracker: incident management REST API

File	Purpose
incident-tracker/Dockerfile	Multi-stage Maven build, JRE 21 runtime as non-root user, port 8081.
incident-tracker/.dockerignore	Excludes build output from the Docker context.
incident-tracker/pom.xml	Spring Boot web, data-jpa, security, validation, actuator, Flyway, PostgreSQL driver; H2 for tests.
incident-tracker/…/incidents/IncidentTrackerApplication.java	Spring Boot entry point.
incident-tracker/…/incidents/config/ApiKeyAuthenticationFilter.java	Authenticates the detection service via X-API-Key using a constant-time comparison; grants ROLE_SERVICE.
incident-tracker/…/incidents/config/SecurityConfig.java	Stateless security: HTTP Basic analysts (ROLE_ANALYST) + API key; SERVICE may only create and read incidents.
incident-tracker/…/incidents/config/TrackerSecurityProperties.java	Validated API key (min 16 chars) and analyst accounts.
incident-tracker/…/incidents/domain/HistoryAction.java	Audit actions: CREATED, STATUS_CHANGED, ASSIGNED, NOTE_ADDED, SEVERITY_CHANGED, RETRIGGERED.
incident-tracker/…/incidents/domain/Incident.java	JPA aggregate root with notes, history, timestamps, detection count and optimistic-lock version.
incident-tracker/…/incidents/domain/IncidentHistory.java	Append-only audit row (action, from, to, actor, timestamp).
incident-tracker/…/incidents/domain/IncidentNote.java	Investigation note (author, content, createdAt).
incident-tracker/…/incidents/domain/IncidentSource.java	DETECTION (from detection-service) or MANUAL (analyst).
incident-tracker/…/incidents/domain/IncidentStatus.java	Lifecycle state machine and the allowed transitions between states.

File	Purpose
incident-tracker/…/incidents/domain/Severity.java	LOW, MEDIUM, HIGH, CRITICAL.
incident-tracker/…/incidents/dto/AssignRequest.java	Body for PATCH /assign (null/blank un-assigns).
incident-tracker/…/incidents/dto/CreateIncidentRequest.java	Validated body for POST /api/incidents including fingerprint and JSON evidence.
incident-tracker/…/incidents/dto/IncidentDetail.java	Full incident view with notes, history and allowed transitions.
incident-tracker/…/incidents/dto/IncidentFilter.java	Optional list filters (status, severity, assignee, user, IP, rule, text, date range).
incident-tracker/…/incidents/dto/IncidentStats.java	Stats response: counts by status/severity, MTTA, MTTR, top users/IPs/rules.
incident-tracker/…/incidents/dto/IncidentSummary.java	Compact row used in lists.
incident-tracker/…/incidents/dto/NoteRequest.java	Body for adding a note.
incident-tracker/…/incidents/dto/PageResponse.java	Stable JSON page wrapper.
incident-tracker/…/incidents/dto/SeverityUpdateRequest.java	Body for PATCH /severity.
incident-tracker/…/incidents/dto/StatusUpdateRequest.java	Body for PATCH /status (status, resolution, comment).
incident-tracker/…/incidents/repository/IncidentRepository.java	Spring Data repository: fingerprint lookup for de-duplication and aggregate queries for stats.
incident-tracker/…/incidents/repository/IncidentSpecifications.java	JPA Criteria builder for the list filters (with LIKE escaping).
incident-tracker/…/incidents/service/CreateResult.java	Result of create: the incident and whether it was de-duplicated.
incident-tracker/…/incidents/service/IncidentNotFoundException.java	Mapped to HTTP 404.
incident-tracker/…/incidents/service/IncidentService.java	Business logic: create/de-duplicate, search, status workflow, assignment, severity, notes, audit history.
incident-tracker/…/incidents/service/IncidentStatsService.java	Computes dashboard statistics including mean time to acknowledge/resolve.
incident-tracker/…/incidents/service/InvalidTransitionException.java	Mapped to HTTP 409 for illegal state changes.
incident-tracker/…/incidents/web/AnalystController.java	GET /api/analysts and /api/me.
incident-tracker/…/incidents/web/ApiExceptionHandler.java	Problem-detail responses for 404, 409 (transition / optimistic lock), 400 validation.
incident-tracker/…/incidents/web/IncidentController.java	All /api/incidents endpoints with paging, sorting whitelist and filters.
incident-tracker/src/main/resources/application.yml	Port 8081, datasource from env, JPA validate mode, Flyway, analyst accounts and API key from env.
incident-tracker/src/main/resources/db/migration/V1__create_incident_schema.sql	Flyway migration creating incidents, incident_notes, incident_history with CHECK constraints and indexes.
incident-tracker/…test/incidents/IncidentApiTest.java	Integration tests (H2): auth, de-duplication, SERVICE restrictions, full analyst workflow, filters and stats.
incident-tracker/src/test/resources/application-test.yml	H2 in PostgreSQL mode plus test credentials.

detection-service: SIEM detection engine

File	Purpose
detection-service/Dockerfile	Multi-stage Maven build, JRE 21 runtime as non-root user, port 8082.
detection-service/.dockerignore	Excludes build output from the Docker context.
detection-service/pom.xml	Spring Boot web, security, validation, actuator (no database).
detection-service/…/detection/DetectionServiceApplication.java	Entry point; enables @Scheduled polling.
detection-service/…/detection/config/AnalystProperties.java	Analyst accounts for the read API (same credentials as the tracker).
detection-service/…/detection/config/DetectionProperties.java	OpenSearch / tracker settings and the list of rules bound from YAML.
detection-service/…/detection/config/HttpClientConfig.java	RestClient instances for OpenSearch (optional Basic auth) and the tracker (X-API-Key), with timeouts.
detection-service/…/detection/config/SecurityConfig.java	HTTP Basic, ROLE_ANALYST for /api/**, health endpoints public.
detection-service/…/detection/engine/AlertStore.java	Persists alerts to the soc-alerts index and queries them for the dashboard.
detection-service/…/detection/engine/DetectionEngine.java	Scheduled loop: evaluate rules, suppress repeats, save alerts, forward to tracker, retry queue, rule status.

File	Purpose
detection-service/…/detection/engine/IncidentForwarder.java	Maps an alert to a CreateIncidentRequest (title, description, fingerprint, evidence) and POSTs it.
detection-service/…/detection/engine/SuppressionCache.java	Per-fingerprint quiet period so an ongoing attack does not alert on every poll.
detection-service/…/detection/opensearch/OpenSearchClient.java	Thin REST wrapper: search (ignoring missing indices), index, update, ping.
detection-service/…/detection/rules/DetectionAlert.java	Fired-alert record; derives fingerprint and title.
detection-service/…/detection/rules/RuleDefinition.java	Rule model: match/exclude, group-by, threshold, window, require, distinct-field, escalation, suppression.
detection-service/…/detection/rules/Severity.java	Alert severities.
detection-service/…/detection/rules/ThresholdRuleQuery.java	Builds the OpenSearch aggregation query for a rule and converts buckets into alerts.
detection-service/…/detection/web/ApiExceptionHandler.java	400 for bad parameters, 502 when OpenSearch is unreachable.
detection-service/…/detection/web/DetectionController.java	/api/alerts, /api/alerts/summary, /api/rules, POST /api/rules/run, /api/events/*, /api/health/pipeline.
detection-service/…/detection/web/EventQueryService.java	Dashboard queries over raw events: recent suspicious events, entity timelines, activity histogram.
detection-service/src/main/resources/application.yml	Poll interval, OpenSearch/tracker settings and the 16 detection rules.
detection-service/…test/detection/ApplicationContextTest.java	Boots the full context with the real rules to catch YAML binding mistakes.
detection-service/…test/detection/rules/ThresholdRuleQueryTest.java	Unit tests for query building, bucket parsing, escalation, distinct and require conditions.
detection-service/src/test/resources/application-test.yml	Test settings (unreachable OpenSearch, long initial delay).

frontend: React SOC dashboard

File	Purpose
frontend/Dockerfile	Builds the Vite bundle with Node 20, serves it from nginx 1.27.
frontend/.dockerignore	Excludes node_modules and dist.
frontend/index.html	HTML shell that loads src/main.jsx.
frontend/nginx.conf	Static hosting, SPA fallback, security headers/CSP, reverse proxy /tracker/ and /detection/.
frontend/package.json	React 18, React Router 6, Vite 5 (pinned versions).
frontend/package-lock.json	Exact dependency tree for reproducible npm ci builds.
frontend/public/shield.svg	Logo / favicon.
frontend/vite.config.js	Dev server on :3000 with the same proxy prefixes as nginx.
frontend/src/main.jsx	Mounts the app with BrowserRouter and AuthProvider.
frontend/src/App.jsx	Login gate and route table.
frontend/src/api.js	fetch wrapper: Basic auth header, JSON errors, 401 → logout; tracker and detection API functions.
frontend/src/auth.jsx	Auth context: validates credentials via /api/me, stores them in sessionStorage.
frontend/src/hooks.js	usePolling: load data and refresh on an interval.
frontend/src/format.js	Severity/status lists and date/time formatting helpers.
frontend/src/styles.css	Dark SOC theme, layout grid, badges, tables, charts.
frontend/src/components/Layout.jsx	Sidebar navigation, SIEM pipeline status, sign-out.
frontend/src/components/ActivityChart.jsx	Stacked SVG column chart of suspicious event types over time.
frontend/src/components/BarList.jsx	Horizontal bar list used for severities, statuses, top users/IPs.
frontend/src/components/Badges.jsx	Severity, status and event-type badges.
frontend/src/components/Card.jsx	Card, StatCard, ErrorBanner, Empty.
frontend/src/components/EventTable.jsx	Event table with pivot links (user, IP, correlation id) and attack type.
frontend/src/pages/Login.jsx	Analyst sign-in form.
frontend/src/pages/Dashboard.jsx	Overview: KPI tiles, activity chart, alerts by severity, status, affected users, top IPs, open incidents, recent events.
frontend/src/pages/Incidents.jsx	Filterable, paged incident list.

File	Purpose
`frontend/src/pages/NewIncidentForm.jsx`	Form to open a manual incident.
`frontend/src/pages/IncidentDetail.jsx`	Triage view: assign, severity, status transitions, evidence, related timeline, notes, audit history.
`frontend/src/pages/Alerts.jsx`	Alert feed and rule catalogue with live status; 'Run detection now'.
`frontend/src/pages/Events.jsx`	Event explorer with pivots by user, IP or correlation id.

Log pipeline, search and database configuration

File	Purpose
`fluent-bit/fluent-bit.conf`	Tail input on app.json, grep filter (audit events + warnings), OpenSearch output to security-events-YYYY.MM.DD, stdout echo.
`fluent-bit/parsers.conf`	JSON parser that uses the app's UTC @timestamp as the event time.
`opensearch/init.sh`	Idempotent init: waits for the cluster, applies templates, creates soc-alerts, ISM policy and the sample monitor.
`opensearch/templates/security-events.json`	Index template: explicit keyword/integer/date mappings for event fields.
`opensearch/templates/soc-alerts.json`	Index template for persisted alerts.
`opensearch/ism-retention-policy.json`	Deletes security-events-* indices after 14 days.
`opensearch/monitors/brute-force-monitor.json`	Example native OpenSearch Alerting bucket monitor (a second, built-in detection layer).
`database/init/01-init.sql`	First-run PostgreSQL script: creates the read-only soc_readonly reporting role.

Scripts and attack simulators

File	Purpose
`scripts/generate-env.sh`	Creates .env with random secrets (Linux/macOS, uses openssl).
`scripts/generate-env.ps1`	Windows PowerShell equivalent of generate-env.sh.
`scripts/simulate-attacks.sh`	All-in-one bash attack script: 7 basic + 9 advanced scenarios.
`scripts/simulate-attacks.ps1`	Windows PowerShell equivalent of simulate-attacks.sh.
`attack-simulators/README.md`	How to run the six standalone simulators and what each should trigger.
`attack-simulators/common.py`	Shared stdlib-only helpers: config from .env, HTTP with spoofed source IP, JWT helpers, random IPs, output.
`attack-simulators/1_credential_stuffing.py`	Botnet: distributed brute force, combo-list replay, admin takeover, bulk data theft.
`attack-simulators/2_vulnerability_scanner.py`	Nikto-style scanner: content discovery, injection fuzzing, ID enumeration.
`attack-simulators/3_insider_threat.py`	Employee bob: admin probing, IDOR enumeration, bulk export.
`attack-simulators/4_session_hijack.py`	Stolen JWT replayed from proxies, claim tampering, alg=none forgery.
`attack-simulators/5_log4shell_campaign.py`	Three bots spraying obfuscated JNDI payloads plus RCE follow-ups.
`attack-simulators/6_ddos_flood.py`	Four hosts flooding the API concurrently with a login flood.

6 Backend 1 - target-app

`target-app` is the monitored system. It plays the role of a production web API, and every security-relevant decision it makes is recorded as a structured event.

6.1 Endpoints

Method & path	Access	Behaviour / events
POST /api/auth/login	public	BCrypt check; returns a JWT. LOGIN_SUCCESS or LOGIN_FAILURE (always the generic message *Invalid username or password*)
GET /api/users/me	any authenticated	Own profile
GET /api/users	ROLE_ADMIN	All users; others get 403 + ACCESS_DENIED
GET /api/orders	authenticated	Own orders (admin: all)
GET /api/orders/{id}	owner or admin	Non-owner: RESOURCE_OWNERSHIP_VIOLATION + 403 (IDOR protection); unknown id: 404
POST /api/orders	authenticated	Validated body; ORDER_CREATED
GET /api/admin/stats, POST /api/admin/maintenance	ROLE_ADMIN	ADMIN_ACTION; others 403 + ACCESS_DENIED (restricted_resource=true)
GET /actuator/health, /actuator/info	public	Container health check (not logged as HTTP_REQUEST)
anything else	denied	401 for anonymous, 403 for authenticated callers

6.2 Request processing pipeline

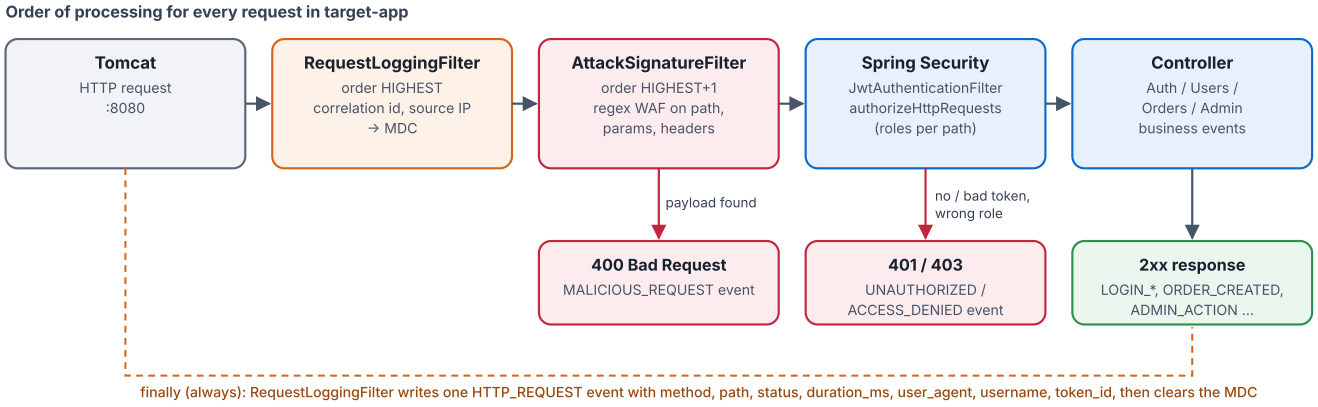


Figure 5 - Filters every request passes through, and the events each outcome produces.

Correlation ID and access event (RequestLoggingFilter)

This filter runs before everything else (`@Order(HIGHEST_PRECEDENCE)`). A client-supplied `X-Correlation-ID` is accepted only if it matches `^[A-Za-z0-9._-]{8,64}$` ; otherwise a UUID is generated. This stops log injection through the header. The ID is returned to the client in the response header and in every error body.

target-app/src/main/java/com/minisoc/target/logging/RequestLoggingFilter.java (lines 41-65)

```
String correlationId = request.getHeader(RequestContext.CORRELATION_HEADER);
if (correlationId == null || !SAFE_ID.matcher(correlationId).matches()) {
    correlationId = UUID.randomUUID().toString();
}
MDC.put(RequestContext.MDC_CORRELATION_ID, correlationId);
MDC.put(RequestContext.MDC_SOURCE_IP, ipResolver.resolve(request));
response.setHeader(RequestContext.CORRELATION_HEADER, correlationId);
try {
    chain.doFilter(request, response);
} finally {
    if (!request.getRequestURI().startsWith("/actuator")) {
        Map<String, Object> details = new LinkedHashMap<>();
        details.put("http_method", request.getMethod());
        details.put("http_path", request.getRequestURI());
        details.put("http_status", response.getStatus());
        details.put("duration_ms", (System.nanoTime() - start) / 1_000_000);
        details.put("user_agent", truncate(request.getHeader("User-Agent")));
        Object user = request.getAttribute(RequestContext.ATTR_USERNAME);
        if (user != null) details.put("username", user.toString());
        events.emit(SecurityEventType.HTTP_REQUEST,
                response.getStatus() < 400 ? "SUCCESS" : "FAILURE",
                "%s %s -> %d".formatted(request.getMethod(), request.getRequestURI(), response.getStatus()),
                details);
    }
    MDC.clear();
```

Authorization rules (SecurityConfig)

target-app/src/main/java/com/minisoc/target/config/SecurityConfig.java (lines 34-49)

```
.csrf(AbstractHttpConfigurer::disable)
.httpBasic(AbstractHttpConfigurer::disable)
.formLogin(AbstractHttpConfigurer::disable)
.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(auth -> auth
        .requestMatchers(HttpMethod.POST, "/api/auth/login").permitAll()
        .requestMatchers("/actuator/health", "/actuator/info").permitAll()
        .requestMatchers("/api/admin/**").hasRole("ADMIN")
        .requestMatchers(HttpMethod.GET, "/api/users").hasRole("ADMIN")
        .requestMatchers("/api/**").authenticated()
        .anyRequest().denyAll())
.exceptionHandling(e -> e
        .authenticationEntryPoint(entryPoint)
        .accessDeniedHandler(accessDeniedHandler))
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
.build();
```

CSRF protection is disabled because the API is stateless and uses bearer tokens, not cookies. There is no session for a cross-site request to ride on. Unauthenticated access goes to `AuditAuthenticationEntryPoint` (401 + UNAUTHORIZED) and insufficient roles to `AuditAccessDeniedHandler` (403 + ACCESS_DENIED).

JWT issue and validation

target-app/src/main/java/com/minisoc/target/security/JwtService.java (lines 30-43)

```
public IssuedToken issue(UserDetails user) {
    Instant now = Instant.now();
    Instant expiry = now.plus(props.jwtTtl());
    List<String> roles = user.getAuthorities().stream().map(GrantedAuthority::getAuthority).toList();
    String token = Jwts.builder()
            .id(UUID.randomUUID().toString())
            .issuer(props.jwtIssuer())
            .subject(user.getUsername())
            .claim("roles", roles)
            .issuedAt(Date.from(now))
            .expiration(Date.from(expiry))
            .signWith(key)
            .compact();
    return new IssuedToken(token, expiry);
```

Tokens are HS256-signed with `JWT_SECRET`. The property is validated to be at least 32 characters at start-up, and the app refuses to start otherwise. Parsing enforces the signature, expiry and issuer. Every token has a random `jti`, which the filter below puts into the MDC as `token_id`. That is what makes the `SESSION_HIJACK` rule possible.

target-app/src/main/java/com/minisoc/target/security/JwtAuthenticationFilter.java (lines 47-69)

```
if (header != null && header.startsWith(BEARER)) {
    try {
        Claims claims = jwtService.parse(header.substring(BEARER.length()));
        String username = claims.getSubject();
        @SuppressWarnings("unchecked")
        List<String> roles = claims.get("roles", List.class);
        var authorities = roles == null ? List.<SimpleGrantedAuthority>of()
                : roles.stream().map(SimpleGrantedAuthority::new).toList();
        var auth = new UsernamePasswordAuthenticationToken(username, null, authorities);
        SecurityContextHolder.getContext().setAuthentication(auth);
        MDC.put(RequestContext.MDC_USERNAME, username);
        if (claims.getId() != null) MDC.put(RequestContext.MDC_TOKEN_ID, claims.getId());
        request.setAttribute(RequestContext.ATTR_USERNAME, username);
    } catch (ExpiredJwtException e) {
        events.failure(SecurityEventType.TOKEN_INVALID, "Expired JWT presented",
                Map.of("reason", "expired", "username", String.valueOf(e.getClaims().getSubject()),
                        "http_path", request.getRequestURI()));
    } catch (JwtException | IllegalArgumentException e) {
        events.failure(SecurityEventType.TOKEN_INVALID, "Invalid JWT presented",
                Map.of("reason", e.getClass().getSimpleName(), "http_path", request.getRequestURI()));
    }
}
chain.doFilter(request, response);
```

Unsigned `alg: none` tokens raise `UnsupportedJwtException` and tampered ones `SignatureException`. Both are logged as TOKEN_INVALID with that class name in `reason`, and the request continues unauthenticated, so it ends in a 401.

Input inspection (AttackSignatureFilter)

A deliberately small WAF that runs right after the logging filter. It inspects the path, every query/form parameter and every header except `Authorization`, `Cookie` and `Content-Length` (at most 4 KB each). The first matching signature wins:

target-app/src/main/java/com/minisoc/target/security/AttackSignatureFilter.java (lines 43-62)

```
private static final List<Signature> PAYLOAD_SIGNATURES = List.of(
        new Signature("jndi_injection", Pattern.compile(
                "\\$\\{[^}]*(jndi|lower:j|upper:j|::-j)", Pattern.CASE_INSENSITIVE)),
        new Signature("sql_injection", Pattern.compile(
                "(\\bunion\\b\\s+(all\\s+)?\\bselect\\b|\\bselect\\b.+\\bfrom\\b|\\binsert\\s+into\\b"
                        + "|\\bdrop\\s+table\\b|'\\s*(or|and)\\s+'?\\w*'?\\s*=\\s*'?\\w*|\\bor\\s+1\\s*=\\s*1\\b"
                        + "|'\\s*;\\s*--|\\b(pg_)?sleep\\s*\\(\\s*\\d+\\s*\\)|;\\s*
(select|drop|delete|update|insert|shutdown)\\b|\\bbenchmark\\s*\\(|\\bwaitfor\\s+delay\\b)",
                Pattern.CASE_INSENSITIVE)),
        new Signature("xss", Pattern.compile(
                "(<\\s*script|javascript:|\\bon(error|load|mouseover)\\s*=|<\\s*(img|svg|iframe)\\b)",
                Pattern.CASE_INSENSITIVE)),
        new Signature("command_injection", Pattern.compile(
                "([;|&]\\s*(cat|ls|id|whoami|uname|wget|curl|nc|bash|sh|powershell)\\b|\\$\\([^)]*\\)|`[^`]+`)",
                Pattern.CASE_INSENSITIVE)),
        new Signature("path_traversal", Pattern.compile(
                "(\\.\\./|\\.\\.\\\\|/etc/(passwd|shadow)|c:\\\\windows)", Pattern.CASE_INSENSITIVE)));

private static final Pattern SCANNER_USER_AGENT = Pattern.compile(
        "(sqlmap|nikto|nmap|masscan|zgrab|gobuster|dirbuster|feroxbuster|wpscan|nuclei|acunetix|nessus|openvas|hydra)",
        Pattern.CASE_INSENSITIVE);
```

target-app/src/main/java/com/minisoc/target/security/AttackSignatureFilter.java (lines 81-94)

```
Optional<Finding> finding = inspect(request);
if (finding.isPresent()) {
    Finding f = finding.get();
    events.failure(SecurityEventType.MALICIOUS_REQUEST, "Blocked request containing " + f.attackType(),
            details(request, f));
    writer.write(request, response, 400, "Bad Request", "Request blocked by input validation");
    return;
}
String userAgent = request.getHeader("User-Agent");
if (userAgent != null && SCANNER_USER_AGENT.matcher(userAgent).find()) {
    events.failure(SecurityEventType.MALICIOUS_REQUEST, "Known security scanner user agent",
            details(request, new Finding("scanner_tool", "header:user-agent", userAgent)));
}
chain.doFilter(request, response);
```

> **Limits of signature matching**
>
> Regex WAFs can be bypassed with encodings and are prone to false positives. Here the filter exists to *generate realistic telemetry*. The real defences are parameterised queries, output encoding and keeping libraries patched. Request bodies are not inspected, because reading them would consume the stream before the controller.

Object-level authorization (IDOR)

target-app/src/main/java/com/minisoc/target/web/OrderController.java (lines 44-48)

```
if (!order.owner().equals(auth.getName()) && !isAdmin(auth)) {
    // Possible IDOR / horizontal privilege escalation attempt.
    events.failure(SecurityEventType.RESOURCE_OWNERSHIP_VIOLATION, "Attempt to read another user's order",
            Map.of("resource", "order", "resource_id", id, "resource_owner", order.owner()));
    throw new AccessDeniedException("Not your order");
```

6.3 Security event logging

All events go through one class, so the schema stays consistent. MDC values (`correlation_id`, `source_ip`, `token_id`) are added by the encoder. `username` is always written by the logger itself, so an event can name the user it is about (for example, the account targeted by a failed login).

target-app/src/main/java/com/minisoc/target/logging/SecurityEventLogger.java (lines 33-56)

```
public void emit(SecurityEventType type, String outcome, String message, Map<String, ?> details) {
    Map<String, Object> fields = new LinkedHashMap<>();
    fields.put("event_type", type.name());
    fields.put("event_category", type.category());
    fields.put("outcome", outcome);
    fields.put("log_type", "security_audit");
    if (details != null) {
        details.forEach((k, v) -> {
            if (v != null) fields.put(k, v);
        });
    }
    // correlation_id and source_ip come from the MDC via the encoder. username is
    // excluded from the MDC output (see logback-spring.xml) and always written here,
    // so events that name a user explicitly (e.g. a failed login) win over the MDC.
    if (!fields.containsKey("username")) {
        String mdcUser = MDC.get(RequestContext.MDC_USERNAME);
        fields.put("username", mdcUser != null ? mdcUser : "anonymous");
    }
    List<Object> args = new ArrayList<>();
    args.add(entries(fields));
    if ("FAILURE".equals(outcome)) {
        AUDIT.warn(message, args.toArray());
    } else {
        AUDIT.info(message, args.toArray());
```

target-app/src/main/resources/logback-spring.xml (lines 27-46)

```
<appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
    <file>${LOG_DIR}/app.json</file>
    <rollingPolicy class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
        <fileNamePattern>${LOG_DIR}/app.%d{yyyy-MM-dd}.%i.json</fileNamePattern>
        <maxFileSize>50MB</maxFileSize>
        <maxHistory>7</maxHistory>
        <totalSizeCap>500MB</totalSizeCap>
    </rollingPolicy>
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
        <timestampPattern>yyyy-MM-dd'T'HH:mm:ss.SSS'Z'</timestampPattern>
        <timeZone>UTC</timeZone>
        <includeContext>false</includeContext>
        <customFields>${JSON_FIELDS}</customFields>
        <excludeMdcKeyName>username</excludeMdcKeyName>
        <fieldNames>
            <levelValue>[ignore]</levelValue>
            <version>[ignore]</version>
        </fieldNames>
    </encoder>
</appender>
```

The file appender rotates at 50 MB, keeps 7 days and caps the total at 500 MB. It is wrapped in an `AsyncAppender` with `neverBlock=true`, so slow disk I/O can never stall request threads.

Event catalogue

event_type	Category	Emitted by	Key extra fields
LOGIN_SUCCESS / LOGIN_FAILURE	authentication	AuthController	username, roles / reason
TOKEN_INVALID	authentication	JwtAuthenticationFilter	reason (SignatureException, ExpiredJwtException, UnsupportedJwtException, MalformedJwtException)
UNAUTHORIZED	authentication	AuditAuthenticationEntryPoint	http_method, http_path
ACCESS_DENIED	authorization	AuditAccessDeniedHandler	http_path, roles, restricted_resource
RESOURCE_OWNERSHIP_VIOLATION	authorization	OrderController	resource_id, resource_owner
MALICIOUS_REQUEST	attack	AttackSignatureFilter	attack_type, attack_location, attack_sample
ADMIN_ACTION	privileged	AdminController	action

event_type	Category	Emitted by	Key extra fields
ORDER_CREATED	business	OrderController	resource_id, amount
HTTP_REQUEST	access	RequestLoggingFilter	http_method, http_path, http_status, duration_ms, user_agent

7 Log pipeline - Fluent Bit & OpenSearch

7.1 Fluent Bit

target-app and fluent-bit share the named volume `target-logs`: read-write for the app, read-only for Fluent Bit. Fluent Bit tails `/var/log/target-app/app.json`, parses each line as JSON, keeps only audit events and warnings/errors, and bulk-indexes them into a daily index.

fluent-bit/fluent-bit.conf (lines 15-61)

```
[INPUT]
    Name              tail
    Tag               app.target
    Path              /var/log/target-app/app.json
    Parser            app_json
    DB                /fluent-bit/state/tail.db
    Refresh_Interval  2
    Read_from_Head    On
    Skip_Long_Lines   On
    Mem_Buf_Limit     20MB
    storage.type      filesystem

# Drop noisy framework start-up logs; keep audit events, access logs and warnings/errors.
[FILTER]
    Name              grep
    Match             app.*
    Logical_Op        or
    # Note: exactly one space between field name and pattern - extra spaces become
    # part of the regex and nothing matches.
    Regex             log_type ^security_audit$
    Regex             level ^(WARN|ERROR)$

[FILTER]
    Name              modify
    Match             app.*
    Add               pipeline fluent-bit

[OUTPUT]
    Name                opensearch
    Match               app.*
    Host                ${OPENSEARCH_HOST}
    Port                ${OPENSEARCH_PORT}
    # With the OpenSearch security plugin enabled, add:
    #   HTTP_User   ${OPENSEARCH_USER}
    #   HTTP_Passwd ${OPENSEARCH_PASSWORD}
    #   tls         On
    tls                 Off
    Logstash_Format     On
    Logstash_Prefix     security-events
    Logstash_DateFormat %Y.%m.%d
    Time_Key            @timestamp
    Time_Key_Nanos      Off
    Suppress_Type_Name  On
    Replace_Dots        On
    Generate_ID         On
    Retry_Limit         False
    Trace_Error         On
```

fluent-bit/parsers.conf (lines 1-7)

```
[PARSER]
    Name         app_json
    Format       json
    # Use the application's own timestamp (UTC, millisecond precision) as the event time.
    Time_Key     @timestamp
    Time_Format  %Y-%m-%dT%H:%M:%S.%LZ
    Time_Keep    Off
```

- **Exactly-once-ish delivery:** file offsets are stored in `tail.db` and buffers on disk (`storage.type filesystem`) in the `fluent-bit-state` volume, so restarts neither lose nor re-send events. `Retry_Limit False` retries forever while OpenSearch is down.
- **Event time is the application's time:** the parser uses the app's UTC `@timestamp`, not the time of shipping. That keeps detection windows accurate even after a backlog is flushed.
- **Gotcha:** in the `grep` filter, put exactly one space between field and regex. Extra spaces become part of the pattern and silently drop everything (see chapter 17).

7.2 OpenSearch

OpenSearch runs as a single node with a 512 MB heap (configurable with `OPENSEARCH_HEAP`). The one-shot `opensearch-init` container waits for a *yellow* cluster and then applies:

Object	File	Purpose
Index template security-events	templates/security-events.json	Explicit mappings: keyword for exact-match/aggregation fields, integer http_status, long duration_ms, date @timestamp; any other string becomes keyword (dynamic template)
Index template soc-alerts	templates/soc-alerts.json	Alert documents; evidence samples stored but not indexed (enabled: false)
Index soc-alerts	init.sh	Created up-front so the dashboard works before the first alert
ISM policy	ism-retention-policy.json	Deletes security-events-* indices older than 14 days
Alerting monitor	monitors/brute-force-monitor.json	Native OpenSearch bucket monitor (>= 5 failures per user in 2 min), a second detection layer visible in OpenSearch Dashboards

opensearch/templates/security-events.json (lines 11-34)

```
"dynamic_templates": [
  {
    "strings_as_keywords": {
      "match_mapping_type": "string",
      "mapping": { "type": "keyword", "ignore_above": 512 }
    }
  }
],
"properties": {
  "@timestamp":      { "type": "date" },
  "message":         { "type": "text", "fields": { "keyword": { "type": "keyword", "ignore_above": 512 } } },
  "service":         { "type": "keyword" },
  "level":           { "type": "keyword" },
  "logger_name":     { "type": "keyword" },
  "log_type":        { "type": "keyword" },
  "event_type":      { "type": "keyword" },
  "event_category":  { "type": "keyword" },
  "outcome":         { "type": "keyword" },
  "username":        { "type": "keyword" },
  "source_ip":       { "type": "keyword" },
  "correlation_id":  { "type": "keyword" },
  "http_method":     { "type": "keyword" },
  "http_path":       { "type": "keyword" },
  "http_status":     { "type": "integer" },
```

opensearch/init.sh (lines 14-20)

```
put() {
  # $1 = path, $2 = file
  code=$(curl -sS -o /tmp/resp.json -w '%{http_code}' -X PUT "${OS_URL}$1" \
    -H 'Content-Type: application/json' --data-binary "@$2")
  echo "PUT $1 -> ${code}"
  case "$code" in 2*) ;; *) cat /tmp/resp.json; echo ;; esac
}
```

Indices use 1 shard and 0 replicas (single node, so the cluster health is green) and a 2-second refresh interval, so new events are searchable almost immediately.

8 Backend 2 - detection-service

The detection service is the analysis half of the SIEM. It has no database of its own: rules are configuration, events and alerts live in OpenSearch, and incidents are handed to the tracker.

8.1 Rule model

Each rule says: *count events that match* `match` *(and not* `exclude` *), grouped by* `group-by`*, over the last* `window`*; fire when a group reaches* `threshold`. Two optional conditions make richer detections possible without code:

- **Cardinality** (`distinct-field` + `min-distinct`): the group must contain at least N different values of a field. For example, one username attacked from five different IPs, or one IP hitting ten different paths.
- **Correlation** (`require`): the group must *also* contain at least one event of another kind. The threshold counts only the `match` events. Example: at least 5 failures plus at least 1 success from the same IP means an account takeover.

detection-service/src/main/java/com/minisoc/detection/rules/RuleDefinition.java (lines 27-45)

```
public record RuleDefinition(
        @NotBlank String id,
        @NotBlank String name,
        String description,
        @NotNull Severity severity,
        @NotNull Duration window,
        @Min(1) int threshold,
        @NotBlank String groupBy,
        Map<String, List<String>> match,
        Map<String, List<String>> exclude,
        String pathPrefix,
        Integer escalateAt,
        Severity escalateTo,
        Duration suppress,
        String mitre,
        Boolean enabled,
        Map<String, List<String>> require,
        String distinctField,
        Integer minDistinct) {
```

Threshold rule (application.yml)

```
- id: BRUTE_FORCE_LOGIN
  name: Potential brute-force login
  description: Repeated failed logins against the same account in a
short period.
  severity: HIGH
  window: 2m
  threshold: 5
  escalate-at: 20
  escalate-to: CRITICAL
  group-by: username
  match:
    event_type: [LOGIN_FAILURE]
  mitre: T1110.001
```

Correlation rule (application.yml)

```
- id: ACCOUNT_TAKEOVER
  name: Successful login after repeated failures
  description: >-
    A source IP failed to log in repeatedly and then succeeded - the
password was
    probably guessed or stuffed. Treat the account as compromised.
  severity: CRITICAL
  window: 10m
  threshold: 5
  group-by: source_ip
  match:
    event_type: [LOGIN_FAILURE]
  require:
    event_type: [LOGIN_SUCCESS]
  mitre: T1110 / T1078
```

8.2 From rule to OpenSearch query

`ThresholdRuleQuery.build` turns a rule into a `size: 0` search with a time-range filter, terms filters and a `terms` aggregation on the group-by field. Each bucket has sub-aggregations for first/last seen, top source IPs, usernames and paths, plus the five newest sample events used as evidence. When a rule has `require`, events matching either side are fetched and counted separately:

detection-service/src/main/java/com/minisoc/detection/rules/ThresholdRuleQuery.java (lines 57-70)

```
if (hasRequire) {
    // Fetch events matching either side, then count them separately per entity:
    // the threshold applies to "matched" only, and "required" must be >= 1.
    bool.put("should", List.of(
            Map.of("bool", Map.of("filter", matchClauses)),
            Map.of("bool", Map.of("filter", requireClauses))));
    bool.put("minimum_should_match", 1);
    subAggs.put("matched", Map.of("filter", Map.of("bool", Map.of("filter", matchClauses))));
    subAggs.put("required", Map.of("filter", Map.of("bool", Map.of("filter", requireClauses))));
    terms.put("min_doc_count", 1);
    terms.put("order", Map.of("matched", "desc"));
} else {
    terms.put("min_doc_count", rule.threshold());
    terms.put("order", Map.of("_count", "desc"));
```

detection-service/src/main/java/com/minisoc/detection/rules/ThresholdRuleQuery.java (lines 90-99)

```
boolean hasRequire = !rule.require().isEmpty();
long count = hasRequire ? bucket.path("matched").path("doc_count").asLong()
        : bucket.path("doc_count").asLong();
if (count < rule.threshold()) continue;
Long distinct = null;
if (rule.hasDistinctCondition()) {
    distinct = bucket.path("distinct").path("value").asLong();
    if (distinct < rule.minDistinct()) continue;
}
if (hasRequire && bucket.path("required").path("doc_count").asLong() == 0) continue;
```

8.3 The engine loop

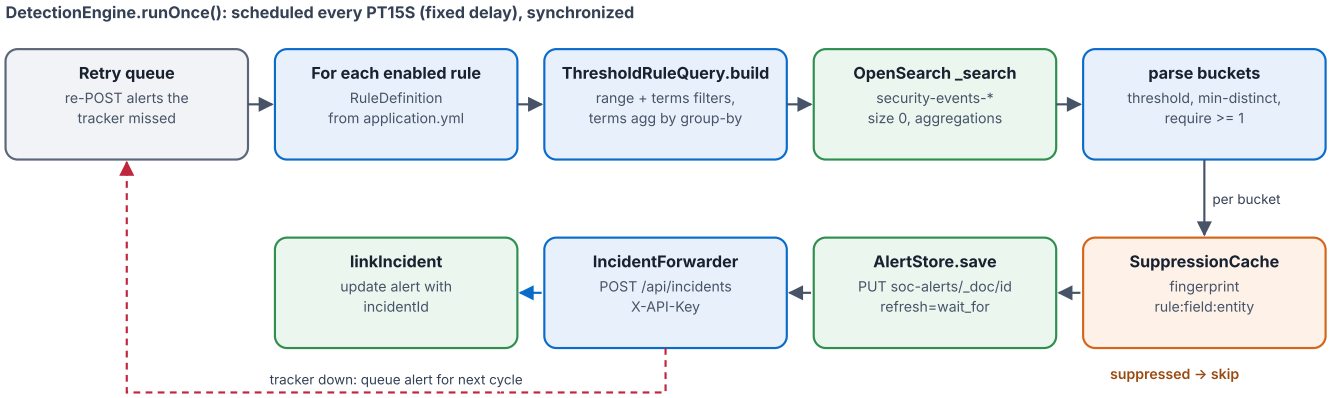


Figure 6 - One detection cycle.

detection-service/src/main/java/com/minisoc/detection/engine/DetectionEngine.java (lines 57-80)

```
public synchronized List<DetectionAlert> runOnce() {
    retryPending();
    Instant now = Instant.now();
    List<DetectionAlert> fired = new ArrayList<>();
    for (RuleDefinition rule : props.rules()) {
        if (!rule.enabled()) continue;
        RuleStatus previous = ruleStatus.getOrDefault(rule.id(), new RuleStatus(null, 0, 0, null));
        try {
            var response = openSearch.search(props.opensearch().eventIndex(), ThresholdRuleQuery.build(rule, now));
            List<DetectionAlert> matches = ThresholdRuleQuery.parse(rule, response, now);
            int newAlerts = 0;
            for (DetectionAlert alert : matches) {
                if (!suppression.tryAcquire(alert.fingerprint(), rule.suppress(), now)) continue;
                handle(alert);
                fired.add(alert);
                newAlerts++;
            }
            ruleStatus.put(rule.id(), new RuleStatus(now, matches.size(), previous.totalFired() + newAlerts, null));
        } catch (Exception e) {
            log.warn("Rule {} failed: {}", rule.id(), e.getMessage());
            ruleStatus.put(rule.id(), new RuleStatus(now, 0, previous.totalFired(), e.getMessage()));
        }
    }
    return fired;
```

- **Suppression:** after firing, a fingerprint (`RULE:field:entity`) stays quiet for `suppress`, which defaults to the window. This keeps one attack from creating an alert on every poll.
- **Resilience:** a failing rule is logged and recorded in its status (shown on the dashboard) without stopping the others. If the tracker is unreachable, alerts wait in an in-memory retry queue that is retried at the start of the next cycle. Searches use `ignore_unavailable`, so rules work before the first event exists.
- **Manual trigger:** `POST /api/rules/run` runs a cycle immediately. It is `synchronized` with the scheduled run, so the two never overlap.

8.4 All 16 rules

Rule	Severity	Condition	Group by	Threshold	MITRE
BRUTE_FORCE_LOGIN Potential brute-force login	HIGH → CRITICAL at 20	event_type in [LOGIN_FAILURE]	username	≥ 5 in 2m	T1110.001
PASSWORD_SPRAY Password spraying from single source	HIGH	event_type in [LOGIN_FAILURE]	source_ip	≥ 10 in 5m	T1110.003
AUTH_ERROR_SPIKE Spike in 401/403 responses	MEDIUM → HIGH at 60	event_type in [HTTP_REQUEST] AND http_status in [401, 403]	source_ip	≥ 15 in 5m	-
HIGH_REQUEST_RATE Unusual request volume	MEDIUM → HIGH at 500	event_type in [HTTP_REQUEST]	source_ip	≥ 100 in 1m	T1498
RESTRICTED_ENDPOINT_ACCESS Attempted access to admin endpoints	HIGH	event_type in [ACCESS_DENIED] AND http_path starts with /api/admin	username	≥ 3 in 5m	T1068
IDOR_ATTEMPT Access to other users' resources	HIGH	event_type in [RESOURCE_OWNERSHIP_VIOLATION]	username	≥ 3 in 10m	T1087
INVALID_TOKEN_ABUSE Repeated invalid JWTs	MEDIUM	event_type in [TOKEN_INVALID]	source_ip	≥ 10 in 5m	T1550.001
ACCOUNT_TAKEOVER Successful login after repeated failures	CRITICAL	event_type in [LOGIN_FAILURE] requires event_type in [LOGIN_SUCCESS]	source_ip	≥ 5 in 10m	T1110 / T1078
DISTRIBUTED_BRUTE_FORCE Distributed brute force on one account	HIGH	event_type in [LOGIN_FAILURE] distinct source_ip ≥ 5	username	≥ 5 in 10m	T1110.004
SESSION_HIJACK Session token used from multiple IPs	HIGH	event_type in [HTTP_REQUEST] AND outcome in [SUCCESS] distinct source_ip ≥ 3	token_id	≥ 3 in 10m	T1550.001
INJECTION_ATTEMPT Injection payloads in requests	HIGH → CRITICAL at 20	event_type in [MALICIOUS_REQUEST] AND attack_type in [sql_injection, xss, path_traversal, command_injection]	source_ip	≥ 3 in 5m	T1190
LOG4SHELL_EXPLOIT Log4Shell (JNDI) exploitation attempt	CRITICAL	event_type in [MALICIOUS_REQUEST] AND attack_type in [jndi_injection]	source_ip	≥ 1 in 10m	T1190

Rule	Severity	Condition	Group by	Threshold	MITRE
SCANNER_TOOL Security scanner detected	MEDIUM	event_type in [MALICIOUS_REQUEST] AND attack_type in [scanner_tool]	source_ip	≥ 5 in 5m	T1595.002
RECON_PATH_SCAN Directory / endpoint discovery	MEDIUM	event_type in [HTTP_REQUEST] AND http_status in [401, 403, 404] distinct http_path ≥ 10	source_ip	≥ 10 in 2m	T1595.003
DATA_SCRAPING Bulk data access by one user	HIGH	event_type in [HTTP_REQUEST] AND http_method in [GET] AND http_status in [200] AND http_path starts with /api/orders	username	≥ 40 in 1m	T1213
JWT_ALG_NONE Unsigned JWT (alg=none) forgery	HIGH	event_type in [TOKEN_INVALID] AND reason in [UnsupportedJwtException]	source_ip	≥ 1 in 10m	T1606.001

8.5 API used by the dashboard

Endpoint	Returns
GET /api/alerts?size&severity&ruleId	Newest alerts from soc-alerts
GET /api/alerts/summary?hours=24	Counts by severity and by rule
GET /api/rules	Every rule definition plus live status (last run, matches, total fired, last error)
POST /api/rules/run	Runs detection now; returns the alerts fired
GET /api/events/recent?size&minutes	Newest suspicious events
GET /api/events/timeline?username|sourceIp|correlationId	All events of one entity, oldest first
GET /api/events/activity?minutes	Histogram of suspicious event types, top users/IPs, status codes
GET /api/health/pipeline	OpenSearch reachability and pending incident forwards

All endpoints require an analyst login (ROLE_ANALYST, HTTP Basic). Query sizes and time windows are clamped (for example, at most 500 alerts or 7 days), so one request cannot overload OpenSearch.

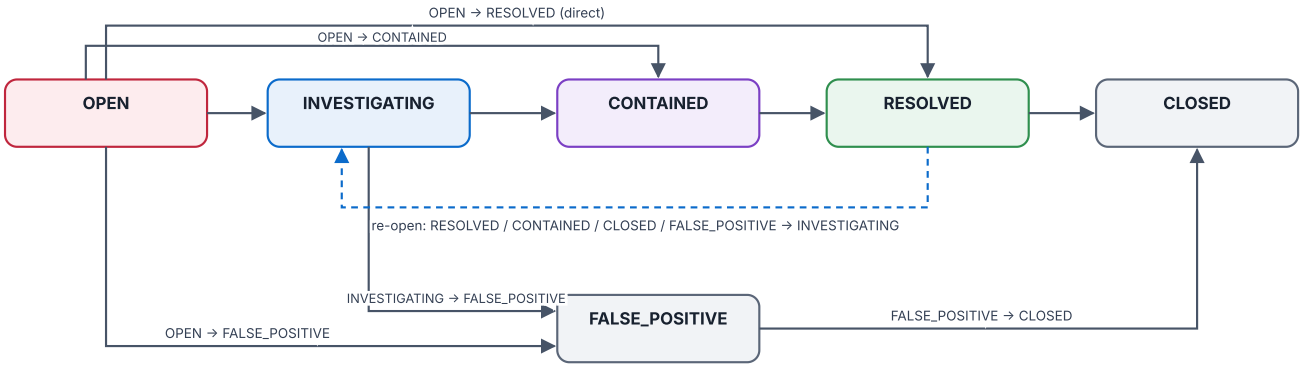
9 Backend 3 - incident-tracker

The incident tracker is a classic layered Spring Boot service: **controller** (HTTP, validation, paging), **service** (business rules, transactions), **repository** (Spring Data JPA + Criteria specifications) and **domain** (JPA entities). Schema changes are owned by Flyway, and Hibernate only *validates* the schema (`ddl-auto: validate`).

9.1 REST API

Method	Endpoint	Who	Function
POST	/api/incidents	SERVICE, ANALYST	Create; 201 + Location. With an active incident of the same fingerprint: 200 + X-Incident-Deduplicated: true
GET	/api/incidents	SERVICE, ANALYST	List/filter: status, severity (repeatable), assignee (or 'unassigned'), affectedUser, sourceIp, ruleId, q, from, to, page, size (max 200), sort (whitelist), direction
GET	/api/incidents/{{id}}	SERVICE, ANALYST	Detail with evidence JSON, notes, history and allowedTransitions
PATCH	/api/incidents/{{id}}/status	ANALYST	{{status, resolution?, comment?}}; 409 if the transition is not allowed or the resolution is missing
PATCH	/api/incidents/{{id}}/assign	ANALYST	{{assignee}}; null or blank un-assigns
PATCH	/api/incidents/{{id}}/severity	ANALYST	{{severity}}
POST	/api/incidents/{{id}}/notes	ANALYST	{{content}}; 201
GET	/api/incidents/stats	SERVICE, ANALYST	Totals, by status/severity, active by severity, unassigned, last 24h, MTTA, MTTR, top users/IPs/rules
GET	/api/analysts, /api/me	SERVICE, ANALYST	Analyst directory (assignment drop-down) and current principal

9.2 Incident lifecycle



Moving to RESOLVED or FALSE_POSITIVE requires a resolution summary (HTTP 409 otherwise). Each change is written to incident_history.

Figure 7 - Allowed status transitions (IncidentStatus.allowedTransitions).

incident-tracker/src/main/java/com/minisoc/incidents/domain/IncidentStatus.java (lines 18-26)

```
public Set<IncidentStatus> allowedTransitions() {
    return switch (this) {
        case OPEN -> EnumSet.of(INVESTIGATING, CONTAINED, RESOLVED, FALSE_POSITIVE);
        case INVESTIGATING -> EnumSet.of(CONTAINED, RESOLVED, FALSE_POSITIVE);
        case CONTAINED -> EnumSet.of(INVESTIGATING, RESOLVED);
        case RESOLVED -> EnumSet.of(CLOSED, INVESTIGATING);
        case FALSE_POSITIVE -> EnumSet.of(CLOSED, INVESTIGATING);
        case CLOSED -> EnumSet.of(INVESTIGATING);
    };
```

incident-tracker/src/main/java/com/minisoc/incidents/service/IncidentService.java (lines 101-129)

```
public IncidentDetail updateStatus(long id, StatusUpdateRequest req, String actor) {
    Incident incident = load(id);
    IncidentStatus from = incident.getStatus();
    IncidentStatus to = req.status();
```

```
    if (from == to) {
        throw new InvalidTransitionException("Incident is already " + to);
    }
    if (!from.canTransitionTo(to)) {
        throw new InvalidTransitionException("Cannot move incident from %s to %s (allowed: %s)"
                .formatted(from, to, from.allowedTransitions()));
    }
    if (to.requiresResolution() && (req.resolution() == null || req.resolution().isBlank())) {
        throw new InvalidTransitionException("A resolution summary is required to mark an incident " + to);
    }

    incident.setStatus(to);
    incident.addHistory(HistoryAction.STATUS_CHANGED, from.name(), to.name(), actor);
    if (incident.getAcknowledgedAt() == null) incident.setAcknowledgedAt(Instant.now());
    if (to.requiresResolution()) {
        incident.setResolution(req.resolution());
        incident.setResolvedAt(Instant.now());
    } else if (to == IncidentStatus.INVESTIGATING && !from.isActive()) {
        // Re-opened: clear the previous resolution timestamp, keep the text for context.
        incident.setResolvedAt(null);
    }
    if (req.comment() != null && !req.comment().isBlank()) {
        incident.addNote(new IncidentNote(actor, "[%s → %s] %s".formatted(from, to, req.comment())));
    }
    return toDetail(repository.save(incident));
```

`acknowledgedAt` is set the first time an incident is assigned or leaves OPEN, and `resolvedAt` when it is resolved. Together they drive the *mean time to acknowledge* and *mean time to resolve* KPIs on the dashboard.

9.3 De-duplication of detections

incident-tracker/src/main/java/com/minisoc/incidents/service/IncidentService.java (lines 51-68)

```
if (req.fingerprint() != null && !req.fingerprint().isBlank()) {
    var existing = repository.findFirstByFingerprintAndStatusInOrderByCreatedAtDesc(
            req.fingerprint(), IncidentStatus.activeStatuses());
    if (existing.isPresent()) {
        Incident incident = existing.get();
        incident.incrementDetectionCount();
        incident.addHistory(HistoryAction.RETRIGGERED, null,
                "detection #" + incident.getDetectionCount(), actor);
        if (req.severity().compareTo(incident.getSeverity()) > 0) {
            incident.addHistory(HistoryAction.SEVERITY_CHANGED, incident.getSeverity().name(),
                    req.severity().name(), actor);
            incident.setSeverity(req.severity());
        }
        if (req.evidence() != null) incident.setEvidence(writeJson(req.evidence()));
        incident.addNote(new IncidentNote(actor,
                "Detection re-triggered: " + Objects.requireNonNullElse(req.description(), req.title())));
        log.info("Incident {} re-triggered by fingerprint {}", incident.getId(), req.fingerprint());
        return new CreateResult(toDetail(repository.save(incident)), true);
```

9.4 Security

incident-tracker/src/main/java/com/minisoc/incidents/config/SecurityConfig.java (lines 30-35)

```
.authorizeHttpRequests(auth -> auth
        .requestMatchers("/actuator/health/**", "/actuator/info").permitAll()
        .requestMatchers(HttpMethod.POST, "/api/incidents").hasAnyRole("SERVICE", "ANALYST")
        .requestMatchers(HttpMethod.GET, "/api/**").hasAnyRole("SERVICE", "ANALYST")
        .requestMatchers("/api/**").hasRole("ANALYST")
        .anyRequest().denyAll())
```

incident-tracker/src/main/java/com/minisoc/incidents/config/ApiKeyAuthenticationFilter.java (lines 32-43)

```
String provided = request.getHeader(HEADER);
if (provided != null) {
    if (MessageDigest.isEqual(expectedKey, provided.getBytes(StandardCharsets.UTF_8))) {
        var auth = new UsernamePasswordAuthenticationToken(SERVICE_PRINCIPAL, null,
                List.of(new SimpleGrantedAuthority("ROLE_SERVICE")));
        SecurityContextHolder.getContext().setAuthentication(auth);
    } else {
        response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Invalid API key");
        return;
    }
}
chain.doFilter(request, response);
```

- `MessageDigest.isEqual` compares the key in constant time, so its prefix cannot be guessed by timing.
- A *wrong* key is rejected immediately with 401. It does not fall through to Basic auth.
- The service principal can create and read but not change incidents: a compromised detection service cannot close incidents to hide an attack.
- The `@Version` column gives optimistic locking. If two analysts edit the same incident at once, the second gets a 409 with "modified by someone else" instead of silently overwriting the first.
- The sort field is checked against a whitelist and the page size is clamped. Free-text search escapes `%` and `_` before building the LIKE pattern.

10 Data stores

10.1 PostgreSQL (incident data)

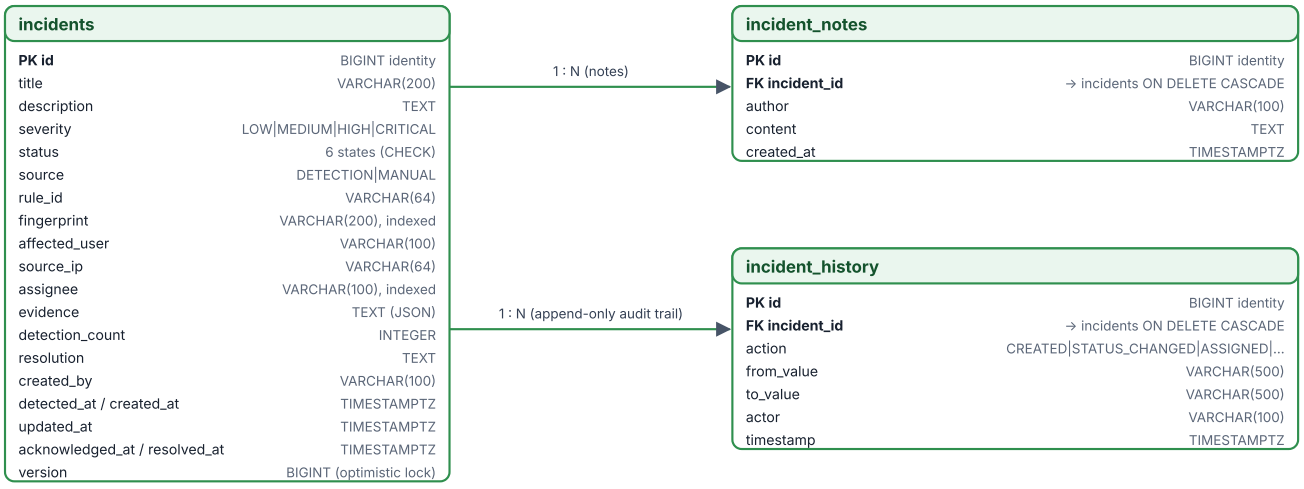


Figure 8 - Incident schema created by Flyway migration V1.

incident-tracker/src/main/resources/db/migration/V1__create_incident_schema.sql (lines 4-28)

```sql
CREATE TABLE incidents (
    id               BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    title            VARCHAR(200) NOT NULL,
    description      TEXT,
    severity         VARCHAR(16)  NOT NULL,
    status           VARCHAR(20)  NOT NULL,
    source           VARCHAR(16)  NOT NULL,
    rule_id          VARCHAR(64),
    fingerprint      VARCHAR(200),
    affected_user    VARCHAR(100),
    source_ip        VARCHAR(64),
    assignee         VARCHAR(100),
    evidence         TEXT,
    detection_count  INTEGER      NOT NULL DEFAULT 1,
    resolution       TEXT,
    created_by       VARCHAR(100) NOT NULL,
    detected_at      TIMESTAMP WITH TIME ZONE,
    created_at       TIMESTAMP WITH TIME ZONE NOT NULL,
    updated_at       TIMESTAMP WITH TIME ZONE NOT NULL,
    acknowledged_at  TIMESTAMP WITH TIME ZONE,
    resolved_at      TIMESTAMP WITH TIME ZONE,
    version          BIGINT       NOT NULL DEFAULT 0,
    CONSTRAINT chk_incident_severity CHECK (severity IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),
    CONSTRAINT chk_incident_status CHECK (status IN ('OPEN', 'INVESTIGATING', 'CONTAINED', 'RESOLVED', 'FALSE_POSITIVE', 'CLOSED')),
    CONSTRAINT chk_incident_source CHECK (source IN ('DETECTION', 'MANUAL'))
```

- **Migrations:** Flyway applies `db/migration/V*.sql` at start-up and records them in `flyway_schema_history`. Never edit an applied migration; add `V2__...sql` instead.
- **Integrity:** CHECK constraints mirror the Java enums. Foreign keys cascade deletes to notes/history. Indexes cover status, severity, created_at, assignee and fingerprint (the de-duplication lookup).
- **Evidence** is stored as JSON text and returned to the client as a parsed JSON object.
- **Persistence:** the `postgres-data` volume. It survives `docker compose down` and is deleted only by `down -v`.
- **First-run script** `database/init/01-init.sql` creates the `soc_readonly` reporting role:

database/init/01-init.sql (lines 8-24)

```
DO $$
BEGIN
    IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = 'soc_readonly') THEN
        CREATE ROLE soc_readonly NOLOGIN;
    END IF;
END
$$;

DO $$
BEGIN
    EXECUTE format('GRANT CONNECT ON DATABASE %I TO soc_readonly', current_database());
END
$$;
GRANT USAGE ON SCHEMA public TO soc_readonly;
-- Tables are created later by Flyway (as the application user); make sure the
-- reporting role can read them.
ALTER DEFAULT PRIVILEGES FOR ROLE CURRENT_USER IN SCHEMA public GRANT SELECT ON TABLES TO soc_readonly;
```

10.2 OpenSearch (events and alerts)

Index	Written by	Read by	Retention
`security-events-YYYY.MM.DD`	Fluent Bit (daily, Logstash_Format)	detection-service rules + event APIs, OpenSearch Dashboards	14 days (ISM)
`soc-alerts`	detection-service AlertStore (id = alertId)	dashboard alert feed and summary	kept until deleted
`.opendistro-*/ .plugins-*`	OpenSearch plugins (ISM, alerting)	internal	-

Data lives in the `opensearch-data` volume. To look at it directly (it is bound to localhost only):

Inspecting OpenSearch from the host

```
curl "localhost:9200/_cat/indices?v"
curl "localhost:9200/security-events-*/_search?q=event_type:LOGIN_FAILURE&size=3&pretty"
curl "localhost:9200/soc-alerts/_search?sort=@timestamp:desc&size=5&pretty"
```

11 Frontend - SOC dashboard

A React 18 single-page app built with Vite 5 and served by nginx. It deliberately uses no UI or chart library: the charts are hand-written SVG and the styling is a single CSS file. The only runtime dependencies are `react`, `react-dom` and `react-router-dom`.

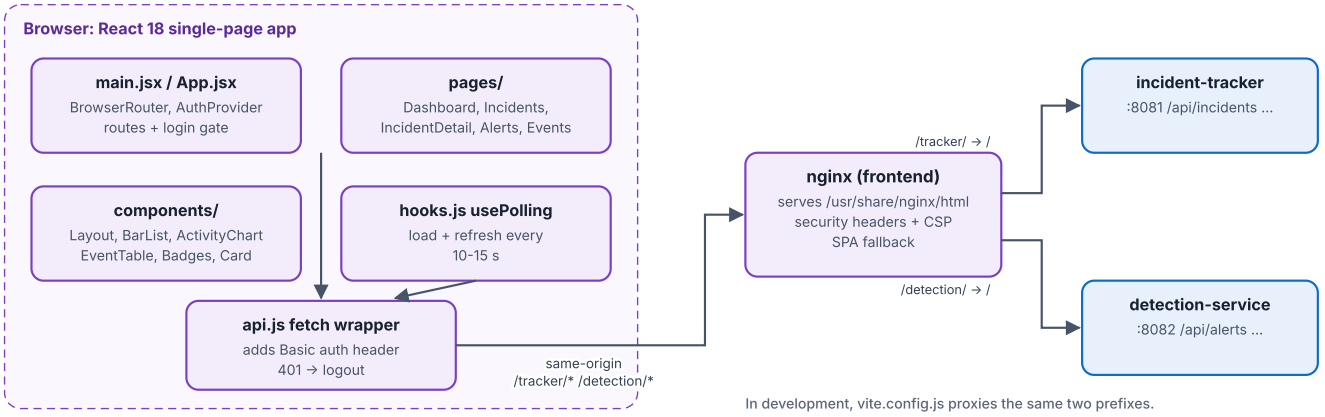


Figure 9 - Frontend structure and how requests reach the backends.

11.1 Pages

Route	Page	Data sources (refresh)
/	Dashboard: KPI tiles, suspicious activity chart, alerts by severity, top rules, incident status, affected users, top IPs, open incidents, recent events	tracker stats + list, detection summary/activity/recent (10 s)
/incidents	Filterable, paged list; chips for status/severity, assignee filter, text search, manual incident form	tracker list (15 s)
/incidents/:id	Triage: assign / assign to me, severity, status transitions with resolution form, evidence and samples, related 24-h timeline, notes, audit history	tracker detail (15 s), detection timeline
/alerts	Alert feed with severity filter, rule catalogue with live status, Run detection now	detection alerts + rules (10-15 s)
/events	Event explorer: pivot by username, source IP or correlation ID; 15 min to 7 days	detection recent/timeline/activity (15 s)

11.2 Authentication and API access

frontend/src/auth.jsx (lines 18-24)

```
const login = useCallback(async (username, password) => {
  const token = btoa(`${username}:${password}`);
  const me = await tracker.me(token);
  const value = { token, username: me.username, roles: me.roles };
  storeAuth(value);
  setAuth(value);
}, []);
```

frontend/src/api.js (lines 32-55)

```
async function request(base, path, { method = 'GET', body, token } = {}) {
  const auth = token ?? getStoredAuth()?.token;
  const res = await fetch(`${base}${path}`, {
    method,
    headers: {
      Accept: 'application/json',
      ...(body !== undefined ? { 'Content-Type': 'application/json' } : {}),
      ...(auth ? { Authorization: `Basic ${auth}` } : {}),
    },
    body: body !== undefined ? JSON.stringify(body) : undefined,
  });
  if (res.status === 401 && !token) onUnauthorized();
  if (!res.ok) {
    let message = `${res.status} ${res.statusText}`;
    try {
      const data = await res.json();
      message = data.detail || data.message || message;
    } catch {
      /* not JSON */
    }
    throw new ApiError(res.status, message);
  }
  if (res.status === 204) return null;
  return res.json();
```

Credentials are kept in `sessionStorage`, which is cleared when the tab closes. Any 401 from a backend logs the user out. Because the API is reached on the same origin, no CORS configuration is needed and the Basic credentials are never sent to another host.

11.3 Polling

frontend/src/hooks.js (lines 4-32)

```
export function usePolling(loader, intervalMs = 0, deps = []) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(true);
  const loaderRef = useRef(loader);
  loaderRef.current = loader;

  const reload = useCallback(async () => {
    try {
      const result = await loaderRef.current();
      setData(result);
      setError(null);
    } catch (e) {
      setError(e);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    setLoading(true);
    reload();
    if (!intervalMs) return undefined;
    const t = setInterval(reload, intervalMs);
    return () => clearInterval(t);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [reload, intervalMs, ...deps]);

  return { data, error, loading, reload };
```

11.4 nginx: static hosting and reverse proxy

frontend/nginx.conf (lines 8-30)

```
add_header X-Content-Type-Options nosniff always;
add_header X-Frame-Options DENY always;
add_header Referrer-Policy no-referrer always;
add_header Content-Security-Policy "default-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; connect-src 'self'" always;

# Backend APIs are reachable only through this proxy (they publish no host ports).
location /tracker/ {
    proxy_pass http://incident-tracker:8081/;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_read_timeout 30s;
}

location /detection/ {
    proxy_pass http://detection-service:8082/;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_read_timeout 30s;
}

# SPA routing.
location / {
    try_files $uri $uri/ /index.html;
```

The trailing slash in `proxy_pass http://incident-tracker:8081/;` strips the prefix, so `/tracker/api/incidents` becomes `/api/incidents`. The Content-Security-Policy only allows scripts, styles and connections to the same origin.

11.5 Build

frontend/Dockerfile (lines 2-13)

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package.json package-lock.json* ./
RUN npm ci --no-audit --no-fund
COPY . .
RUN npm run build

# ---- runtime ----
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

For local development without Docker: `cd frontend && npm install && npm run dev`. Vite serves on :3000 and proxies the two prefixes to localhost:8081 and :8082.

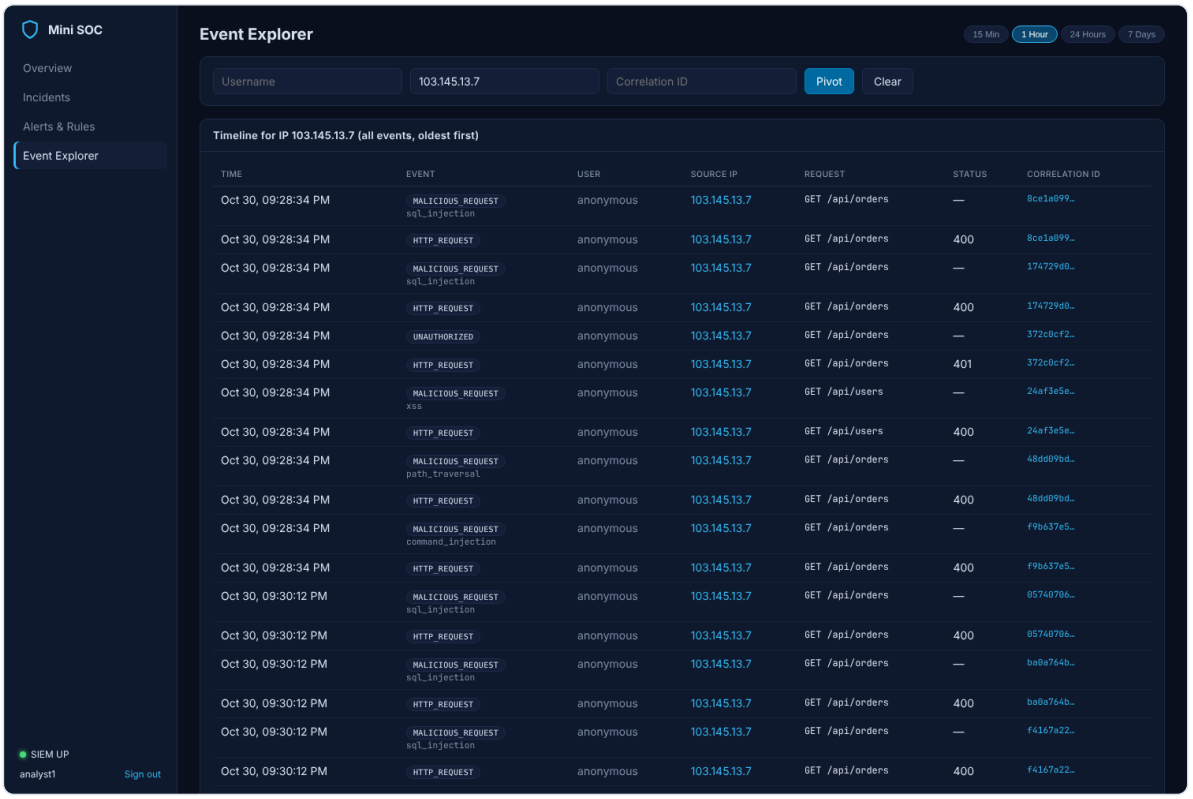


Figure 10 - Event Explorer pivoted on one attacker IP; each blocked payload shows its attack type.

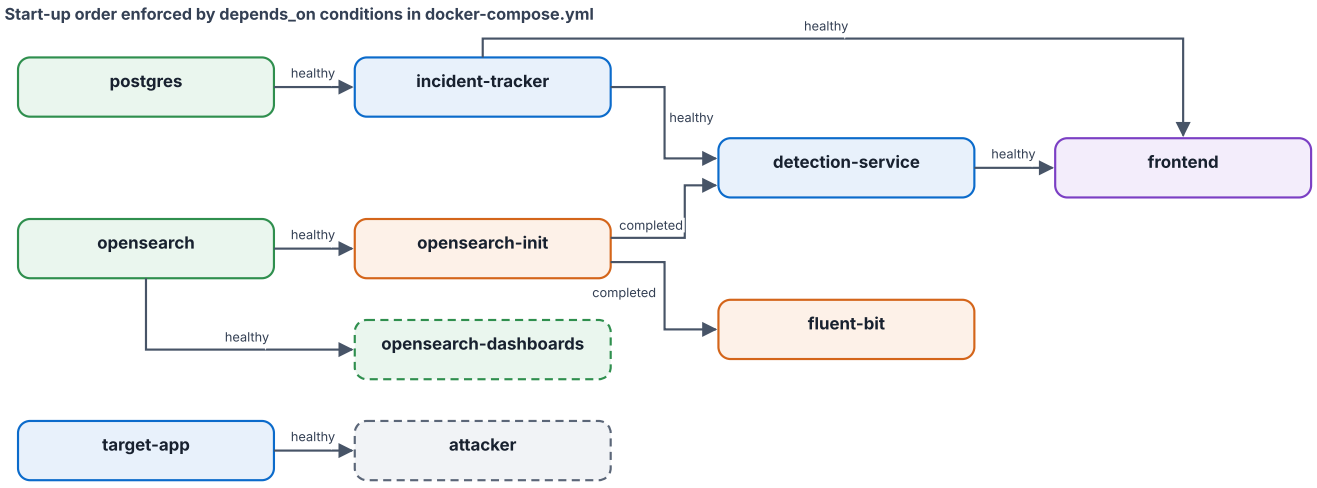
12 Service communication

From	To	Protocol / address	Authentication	Purpose
Clients, simulators	target-app	HTTP host:8080	JWT Bearer (except login)	Use / attack the application
target-app	fluent-bit	File on volume target-logs	filesystem (read-only for Fluent Bit)	Security events
fluent-bit	opensearch	HTTP opensearch:9200 /_bulk	none (lab)	Index events
opensearch-init	opensearch	HTTP opensearch:9200	none (lab)	Templates, ISM, monitor
detection-service	opensearch	HTTP opensearch:9200 _search, _doc, _update	optional Basic (OPENSEARCH_USER)	Run rules, store alerts, event queries
detection-service	incident-tracker	HTTP incident-tracker:8081 POST /api/incidents	X-API-Key	Open / re-trigger incidents
incident-tracker	postgres	JDBC postgres:5432/soc	user/password	Persist incidents
Analyst browser	frontend (nginx)	HTTP host:3000	none for static files	Load the SPA
frontend (nginx)	incident-tracker	/tracker/* → :8081	analyst Basic (passed through)	Incident APIs
frontend (nginx)	detection-service	/detection/* → :8082	analyst Basic (passed through)	Alert / event APIs
attacker container	target-app	HTTP target-app:8080	JWT as the scenario requires	Simulated attacks
Docker health checks	each service	curl localhost/actuator/health, pg_isready, _cluster/health	-	Readiness gating

Inside the network, services address each other by their Compose service name (Docker DNS). Only three ports are published to the host: **8080** (target-app) and **3000** (dashboard) on all interfaces, and **9200** (OpenSearch) and optionally **5601** (OpenSearch Dashboards) on 127.0.0.1 only.

13 Docker deployment & operations

13.1 Compose topology



target-app has no dependencies: it only writes to the shared target-logs volume, so it keeps serving even if the SIEM side is down.
Dashed boxes are optional: attacker (docker compose run) and opensearch-dashboards (--profile tools). opensearch-init exits after applying templates.

Figure 11 - Dependency graph; arrows point from a dependency to the service waiting on it.

Service	Image / build	Volumes	Health check	Restart
postgres	postgres:16-alpine	postgres-data, ./database/init (ro)	pg_isready	unless-stopped
opensearch	opensearchproject/opensearch:2.17.1	opensearch-data	_cluster/health yellow	unless-stopped
opensearch-init	curlimages/curl:8.10.1	./opensearch (ro)	exits 0 when done	no
fluent-bit	fluent/fluent-bit:3.1.9	configs (ro), target-logs (ro), fluent-bit-state	-	unless-stopped
target-app	build ./target-app	target-logs	/actuator/health	unless-stopped
incident-tracker	build ./incident-tracker	-	/actuator/health	unless-stopped
detection-service	build ./detection-service	-	/actuator/health	unless-stopped
frontend	build ./frontend	-	-	unless-stopped
opensearch-dashboards	opensearch-dashboards:2.17.1 (profile tools)	-	-	-
attacker	python:3.12-alpine (profile attacks)	./attack-simulators (ro)	-	run --rm

13.2 Key compose definitions

docker-compose.yml (lines 16-20)

```
x-spring-defaults: &spring-defaults
  restart: unless-stopped
  networks: [soc-net]
  environment: &spring-env
    JAVA_OPTS: "-XX:MaxRAMPercentage=75 -Duser.timezone=UTC"
```

docker-compose.yml (lines 111-132)

```yaml
target-app:
  <<: *spring-defaults
  build: ./target-app
  environment:
    <<: *spring-env
    JWT_SECRET: ${JWT_SECRET:?set JWT_SECRET in .env}
    DEMO_ALICE_PASSWORD: ${DEMO_ALICE_PASSWORD:?set DEMO_ALICE_PASSWORD in .env}
    DEMO_BOB_PASSWORD: ${DEMO_BOB_PASSWORD:?set DEMO_BOB_PASSWORD in .env}
    DEMO_ADMIN_PASSWORD: ${DEMO_ADMIN_PASSWORD:?set DEMO_ADMIN_PASSWORD in .env}
    # Lets the attack simulator spoof X-Forwarded-For to emulate many attackers.
    TRUST_FORWARDED_FOR: ${TRUST_FORWARDED_FOR:-true}
    LOG_DIR: /var/log/target-app
  volumes:
    - target-logs:/var/log/target-app
  ports:
    - "${TARGET_APP_PORT:-8080}:8080"
  healthcheck:
    test: ["CMD-SHELL", "curl -fsS http://localhost:8080/actuator/health >/dev/null"]
    interval: 10s
    timeout: 5s
    retries: 20
    start_period: 30s
```

Required secrets use `${VAR:?message}`. Without a `.env`, Compose refuses to start and names the missing variable, instead of starting services with empty passwords.

13.3 Images

target-app/Dockerfile (lines 1-19)

```dockerfile
# ---- build ----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /build
COPY pom.xml .
RUN mvn -q -B dependency:go-offline
COPY src ./src
RUN mvn -q -B package -DskipTests

# ---- runtime ----
FROM eclipse-temurin:21-jre
RUN groupadd --system app && useradd --system --gid app --no-create-home app \
    && mkdir -p /var/log/target-app && chown app:app /var/log/target-app
WORKDIR /app
COPY --from=build /build/target/app.jar app.jar
USER app
ENV LOG_DIR=/var/log/target-app \
    JAVA_OPTS="-XX:MaxRAMPercentage=75"
EXPOSE 8080
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]
```

- **Multi-stage:** Maven and the JDK only exist in the build stage, and the runtime image contains just the JRE and app.jar.
- **Layer caching:** `pom.xml` is copied and dependencies are resolved before the sources, so code changes do not re-download dependencies.
- **Non-root:** every Spring service runs as the unprivileged `app` user. `exec` makes Java PID 1, so it receives SIGTERM from `docker compose down` and shuts down gracefully.
- **Container-aware memory:** `-XX:MaxRAMPercentage=75` sizes the heap from the container limit.

13.4 Operating the stack

Task	Command
First-time secrets	`./scripts/generate-env.sh` or `powershell -ExecutionPolicy Bypass -File scripts\generate-env.ps1`
Build and start	`docker compose up -d --build`

Task	Command
Start (no code changes)	`docker compose up -d`
Status / health	`docker compose ps`
Follow logs	`docker compose logs -f detection-service`
Rebuild one service	`docker compose up -d --build target-app`
Stop, keep data	`docker compose down`
Stop and delete all data	`docker compose down -v`
Optional OpenSearch Dashboards	`docker compose --profile tools up -d opensearch-dashboards`
Run a simulator in Docker	`docker compose run --rm attacker 1_credential_stuffing.py`
Update code	`git pull` then `docker compose up -d --build`

Changing secrets after the first start

PostgreSQL stores `POSTGRES_PASSWORD` when its volume is first initialised. If you regenerate `.env` later, the tracker can no longer log in to the database. Either keep the old `.env`, or reset the database with `docker compose down -v` before regenerating.

13.5 Windows notes

- Use Docker Desktop with the WSL 2 backend. Open a *new* terminal after installing it, so `docker` is on the PATH.
- Use the `.ps1` scripts. The `.sh` scripts need bash.
- `.gitattributes` keeps LF line endings for files used inside Linux containers. If a clone predates it, run `git rm -r --cached -q . && git reset --hard` once.
- If OpenSearch keeps restarting with a `vm.max_map_count` error, run `wsl -d docker-desktop sysctl -w vm.max_map_count=262144`.
- Docker Desktop keeps RAM reserved (the *Vmmem* process) until you quit it from the tray icon.

14 Security measures & safeguards

The project has two security roles. It **defends** the target application, and it **monitors** it. The table lists every control and where it is implemented.

Control	What it does	Where
Authentication	Signed, expiring JWTs (HS256, 30 min, issuer check, random jti); secret must be at least 32 chars or the app will not start	JwtService, AppSecurityProperties
Password storage	BCrypt hashes, including demo users; timing-safe 'user not found' handling	target-app SecurityConfig
Authorization	Role rules per path (ADMIN for /api/admin/** and the user list); deny-by-default for unknown paths	target-app SecurityConfig
Object-level access	Order ownership check prevents IDOR and is logged	OrderController
Input validation	Bean Validation on all request bodies (sizes, ranges, required fields)	LoginRequest, CreateOrderRequest, tracker DTOs
Attack blocking	Signature WAF for SQLi, XSS, traversal, command injection, Log4Shell; scanner detection	AttackSignatureFilter
Error hygiene	No stack traces or exception messages to clients; RFC 7807 bodies; generic login error (no user enumeration)	application.yml server.error, exception handlers
Log integrity	JSON encoding (no log forging), validated correlation IDs, truncated user agents, separate MDC/explicit username	RequestLoggingFilter, logback-spring.xml
IP spoofing control	X-Forwarded-For only honoured when TRUST_FORWARDED_FOR=true	ClientIpResolver
Service-to-service auth	API key with constant-time comparison; wrong key rejected outright	ApiKeyAuthenticationFilter
Least privilege	Detection service can create and read incidents only; DB reporting role is read-only and NOLOGIN by default	tracker SecurityConfig, 01-init.sql
Analyst access	Both APIs require ROLE_ANALYST; backends publish no host ports; nginx is the only entry	SecurityConfig x2, docker-compose.yml
Browser security	CSP self-only, X-Frame-Options DENY, nosniff, no-referrer; credentials in sessionStorage; 401 → logout	nginx.conf, auth.jsx, api.js
Data integrity	Flyway-managed schema, Hibernate validate, CHECK constraints, FKs, optimistic locking, mandatory resolution, full audit history	V1 migration, Incident, IncidentService
Query safety	JPA Criteria (parameterised), LIKE escaping, sort whitelist, page and window clamps	IncidentSpecifications, controllers
Secrets management	.env git-ignored and chmod 600, random generation, required-variable checks in Compose	.gitignore, generate-env.*, docker-compose.yml
Container hardening	Non-root users, minimal JRE images, read-only mounts for configs and logs, OpenSearch bound to 127.0.0.1	Dockerfiles, docker-compose.yml
Availability	Health-gated start-up, restart policies, async non-blocking logging, file-buffered log shipping with retries	docker-compose.yml, logback, fluent-bit.conf
Detection reliability	Suppression, fingerprint de-duplication, retry queue, per-rule error status, missing-index tolerance	DetectionEngine, IncidentService
Retention	Event indices deleted after 14 days; log files rotated (50 MB, 7 days, 500 MB cap)	ism-retention-policy.json, logback-spring.xml

14.1 Known lab limitations

Limitation	Why	Production fix
OpenSearch security plugin disabled, plain HTTP	Simplicity; bound to 127.0.0.1	Enable the plugin, TLS, users/roles; set OPENSEARCH_USER/PASSWORD and Fluent Bit tls On
HTTP Basic analyst accounts in config	No identity provider needed	OIDC/SSO (Keycloak, Entra ID) with MFA
No TLS between browser and nginx	Local use	TLS termination (certificates) at nginx or a load balancer
Demo users in memory	Self-contained	Real user store; account lockout / rate limiting
X-Forwarded-For trusted	Needed to simulate many attackers	TRUST_FORWARDED_FOR=false, or trust only a known proxy
Suppression state in memory	Single instance	Persist suppression in OpenSearch/Redis if scaled out
Detection only, no automatic response	Keeps the lab safe	SOAR actions: block IP, revoke tokens, disable account

15 Attack simulation & expected outcomes

Each simulator is a self-contained campaign with its own attacker persona and fresh random IPs on every run. They use only the Python standard library.

#	Simulator	What it does	Expected detections
1	1_credential_stuffing.py	7-IP botnet on alice; one bot replays 12 leaked combos then logs in as admin; dumps orders 45x	DISTRIBUTED_BRUTE_FORCE, BRUTE_FORCE_LOGIN, PASSWORD_SPRAY, ACCOUNT_TAKEOVER (CRITICAL), DATA_SCRAPING
2	2_vulnerability_scanner.py	Nikto UA; 30 sensitive paths, 36 injection payloads, 60 ID probes (~126 requests)	SCANNER_TOOL, RECON_PATH_SCAN, INJECTION_ATTEMPT (CRITICAL), HIGH_REQUEST_RATE, AUTH_ERROR_SPIKE
3	3_insider_threat.py	bob works normally, probes admin, reads alice's orders, bulk-exports	RESTRICTED_ENDPOINT_ACCESS, IDOR_ATTEMPT, DATA_SCRAPING
4	4_session_hijack.py	alice's JWT replayed from 5 Tor/proxy IPs; 14 tampered tokens; alg=none forgery	SESSION_HIJACK, INVALID_TOKEN_ABUSE, JWT_ALG_NONE, AUTH_ERROR_SPIKE
5	5_log4shell_campaign.py	3 bots x 5 obfuscated JNDI variants in 5 headers + params, plus RCE droppers	LOG4SHELL_EXPLOIT x3 (CRITICAL), INJECTION_ATTEMPT x3
6	6_ddos_flood.py	4 hosts x 160 concurrent requests incl. 15 random-account logins each	HIGH_REQUEST_RATE x4, AUTH_ERROR_SPIKE x4, PASSWORD_SPRAY x4

Ways to run attacks

```
docker compose run --rm attacker 1_credential_stuffing.py     # inside Docker, no Python needed
python attack-simulators/2_vulnerability_scanner.py           # local Python 3.9+
./scripts/simulate-attacks.sh advanced                         # all-in-one bash script (groups: basic, advanced)
powershell -ExecutionPolicy Bypass -File scripts\simulate-attacks.ps1 takeover hijack
```

15.1 How the simulators fake many attackers

attack-simulators/common.py (lines 60-81)

```python
def request(method, path, ip, token=None, body=None, user_agent=BROWSER_UA, headers=None, query=None):
    """Sends one request as if it came from `ip` and returns (status, parsed JSON body or None)."""
    url = TARGET + path
    if query:
        url += "?" + urllib.parse.urlencode(query)
    data = json.dumps(body).encode() if body is not None else None
    req = urllib.request.Request(url, data=data, method=method)
    req.add_header("X-Forwarded-For", ip)
    req.add_header("User-Agent", user_agent)
    if data is not None:
        req.add_header("Content-Type", "application/json")
    if token:
        req.add_header("Authorization", f"Bearer {token}")
    for name, value in (headers or {}).items():
        req.add_header(name, value)
    try:
        with urllib.request.urlopen(req, timeout=15) as res:
            return res.status, _json(res.read())
    except urllib.error.HTTPError as e:
        return e.code, _json(e.read())
    except urllib.error.URLError as e:
        sys.exit(f"{RED}Cannot reach {TARGET} ({e.reason}). Is the stack running? "
```

attack-simulators/4_session_hijack.py (lines 37-44)

```python
header, _, signature = token.split(".")
claims = jwt_claims(token)
statuses = []
for i in range(14):
    forged = dict(claims, roles=["ROLE_USER", "ROLE_ADMIN"], sub=random.choice(["alice", "admin"]),
                  exp=claims["exp"] + 86400 * (i + 1))
    statuses.append(request("GET", "/api/admin/stats", attacker, token=f"{header}.{b64url(forged)}.{signature}")[0])
report("tampered tokens (401 = signature check failed)", statuses)
```

15.2 Verified outcome

All six simulators were run one after another against the full Docker stack. Every detection each one lists fired, with the correct user or IP attached to the incident. Two behaviours are by design:

- Rules grouped by **IP** create new incidents on every run, because the IPs are random.
- Rules grouped by **user** stay quiet for their window after firing, then add to the existing open incident (its hit count goes up). Resolve the incident to get a fresh one on the next run.

16 Testing

Suite	Type	Tests	Covers
target-app SecurityFlowTest	Spring Boot + MockMvc	6	Bad password 401 + correlation header, missing token 401, non-admin 403 on admin/users, admin 200, IDOR 403, tampered token 401
target-app AttackDetectionTest	Spring Boot + MockMvc	6	SQLi incl. stacked/time-based, XSS, traversal, command injection, Log4Shell (incl. obfuscated), scanner UA passes, benign input passes, alg=none rejected
incident-tracker IncidentApiTest	Spring Boot + H2 (PostgreSQL mode) + Flyway	5	Auth (none / wrong key), fingerprint de-dup + severity escalation, SERVICE cannot PATCH, full analyst workflow + illegal transitions, filters + stats
detection ThresholdRuleQueryTest	Unit	5	Query shape, bucket parsing, escalation, distinct condition, require condition
detection ApplicationContextTest	Spring Boot context	2	All 16 YAML rules bind correctly; API requires login

Running the tests

```
cd target-app && mvn test
cd incident-tracker && mvn test
cd detection-service && mvn test
cd frontend && npm ci && npm run build      # type/bundle check of the dashboard
```

The incident-tracker tests run the real Flyway migration against H2, so a broken migration fails the build. Beyond the unit tests, the stack was verified end to end: build images, start Compose, run the simulators, check alerts, incidents and the dashboard in a headless browser.

17 Troubleshooting guide

Several rows are problems actually hit while building and first running this project: Windows script errors, Docker not on the PATH, the existing-.env check, the missing-sources build failure, CRLF line endings, the Fluent Bit filter spacing and the scheduler duration format. The rest are the most likely problems for this stack.

Symptom	Cause	Fix
`'.' is not recognized` in cmd / PowerShell	A bash script was run on Windows	Use the .ps1 scripts or `docker compose run --rm attacker ...`
`docker : The term 'docker' is not recognized`	Docker Desktop not installed, or the terminal was opened before installing	Install Docker Desktop, start it, open a new terminal; restart if needed
`set POSTGRES_USER in .env` (or another variable)	No .env file	Run generate-env.sh / .ps1
`.env already exists (use -Force)`	Safety check	Keep the existing file; only use -Force together with `docker compose down -v`
Maven build: `Unable to find main class`	Java sources missing from the checkout (an old .gitignore bug, fixed in 241bd6d)	`git pull`, then build again
Maven/npm build: `PKIX path building failed`	A proxy or antivirus intercepts TLS inside Docker builds	Add the corporate CA to the build image, or build on a network without interception
`$'\r': command not found`, opensearch-init fails	Windows CRLF line endings in shell files	`git pull` (adds .gitattributes) then `git rm -r --cached -q . && git reset --hard`
opensearch restarting; log mentions `max_map_count`	Kernel setting too low	`wsl -d docker-desktop sysctl -w vm.max_map_count=262144` (Linux: `sudo sysctl -w ...`)
opensearch exits with code 137	Out of memory	Give Docker more memory, or lower OPENSEARCH_HEAP (for example 384m)
`error setting rlimit ... memlock`	The host forbids the memlock ulimit	Remove `ulimits` and set `bootstrap.memory_lock: "false"` for opensearch
incident-tracker unhealthy; logs show `password authentication failed`	.env regenerated after the database was created	Restore the old .env, or `docker compose down -v` and start again
`port is already allocated`	8080 or 3000 in use	Set TARGET_APP_PORT / DASHBOARD_PORT in .env
Dashboard: Invalid analyst credentials	Wrong password, or .env changed without recreating containers	Use ANALYST1_PASSWORD from .env; `docker compose up -d` to recreate
Sidebar shows SIEM DOWN	OpenSearch unreachable	`docker compose ps opensearch`, `docker compose logs opensearch`
Attacks ran but no alerts	Events not reaching OpenSearch	Check `curl localhost:9200/_cat/indices?v`; Fluent Bit metrics (grep filter drop count); `docker compose logs fluent-bit`
Fluent Bit running but nothing indexed; metrics show grep drop_records equal to records	Extra spaces between field and pattern in a grep Regex line	Use exactly one space (`Regex log_type ^security_audit$`), then reset the fluent-bit-state volume to re-read the file
Events indexed but no incidents	Tracker unreachable or a rule erroring	Alerts & Rules page (rule lastError), `/detection/api/health/pipeline` pending forwards, `docker compose logs detection-service`
Re-running a simulator creates no new incident	Suppression and fingerprint de-dup (by design)	Wait for the rule window, or resolve the existing incident
detection-service crash: `Invalid initialDelayString`	Interval written as 15s	Use ISO-8601 durations (PT15S) for DETECTION_POLL_INTERVAL
PowerShell: script cannot be loaded (execution policy)	Windows blocks downloaded scripts	Run with `powershell -ExecutionPolicy Bypass -File ...`
Simulator: `Cannot reach http://localhost:8080`	Stack not running or target-app unhealthy	`docker compose up -d`, wait for `docker compose ps` to show healthy
Old JWT rejected after restart	JWT_SECRET changed	Log in again (tokens are signed with the secret)

17.1 Diagnostic commands

Where to look

```
docker compose ps                                 # health of every service
docker compose logs --tail 100 fluent-bit         # shipping errors
docker compose logs --tail 100 detection-service  # rule errors, forwarding retries
curl localhost:8080/actuator/health               # target-app
curl "localhost:9200/_cluster/health?pretty"      # OpenSearch
curl "localhost:9200/_cat/indices/security-events-*?v"
docker run --rm --network mini-soc-net curlimages/curl:8.10.1 -s http://fluent-bit:2020/api/v1/metrics
docker compose exec postgres psql -U "$POSTGRES_USER" -d soc -c "select id, severity, status, title from incidents order by id desc limit 10;"
```

17.2 Following one request

Every response from target-app has an `X-Correlation-ID` header, also present in its error bodies. Paste it into **Event Explorer → Correlation ID** to see every event that request produced. Evidence samples in an incident carry the same IDs.

18 Extending & hardening

18.1 Add a detection rule (no code)

1. Append a rule to `detection-service/src/main/resources/application.yml`.
2. Rebuild the service: `docker compose up -d --build detection-service`.
3. Check it on **Alerts & Rules**; a failing query shows its error there.

Example: new rule

```yaml
  - id: OFF_HOURS_ADMIN
    name: Admin activity from a new source
    description: Admin actions from an IP outside the office range.
    severity: HIGH
    window: 10m
    threshold: 1
    group-by: source_ip
    match:
      event_type: [ADMIN_ACTION]
    exclude:
      source_ip: ["10.0.0.10", "10.0.0.11"]
    mitre: T1078
```

Fields must exist as **keyword**-mapped fields in `security-events-*`. New string fields are mapped as keyword automatically by the dynamic template.

18.2 Add a new security event

1. Add a constant to `SecurityEventType` with a category.
2. Call `events.failure(...)` or `events.success(...)` where it happens, passing detail fields.
3. Optionally add explicit mappings to `opensearch/templates/security-events.json` (applied to the next daily index).
4. To show it in the dashboard chart and suspicious lists, add it to `SUSPICIOUS_TYPES` (EventQueryService) and `SERIES` (ActivityChart.jsx).

18.3 Change the incident schema

Add `incident-tracker/src/main/resources/db/migration/V2__describe_change.sql`, update the entity and DTOs, and run `mvn test`. The tests apply all migrations to H2.

18.4 Production checklist

- Enable OpenSearch security (TLS + users) and give the detection service a least-privilege role.
- Put TLS in front of nginx and target-app; replace Basic auth with OIDC and MFA for analysts.
- Store secrets in a secret manager (Docker secrets, Vault, cloud KMS) instead of .env.
- Set `TRUST_FORWARDED_FOR=false`, or trust only your load balancer.
- Add rate limiting and account lockout to the target app (prevention, not just detection).
- Run OpenSearch with replicas and snapshots; back up PostgreSQL (pg_dump or WAL archiving).
- Persist suppression state and run several detection workers with leader election if scaling out.
- Ship the platform's own logs (tracker, detection) to the SIEM too, to monitor the monitor.

A Appendix - API & configuration reference

A.1 Environment variables

Variable	Used by	Meaning
`POSTGRES_USER / POSTGRES_PASSWORD`	postgres, incident-tracker	Database owner account
`JWT_SECRET`	target-app	HMAC key for JWTs (min 32 chars)
`DEMO_ALICE_PASSWORD / DEMO_BOB_PASSWORD / DEMO_ADMIN_PASSWORD`	target-app, simulators	Demo user passwords
`TRUST_FORWARDED_FOR`	target-app	Honour X-Forwarded-For (true in the lab)
`TRACKER_API_KEY`	incident-tracker, detection-service	Service API key (min 16 chars)
`ANALYST1_PASSWORD / SOC_LEAD_PASSWORD`	incident-tracker, detection-service	Analyst logins
`OPENSEARCH_HEAP`	opensearch	JVM heap, default 512m
`DETECTION_POLL_INTERVAL`	detection-service	ISO-8601 duration, default PT15S
`TARGET_APP_PORT / DASHBOARD_PORT`	compose	Published host ports, default 8080 / 3000
`OPENSEARCH_URL / TRACKER_URL`	detection-service (set by compose)	Internal service addresses
`DB_URL / DB_USER / DB_PASSWORD`	incident-tracker (set by compose)	JDBC connection
`LOG_DIR`	target-app (set by compose)	Where app.json is written
`TARGET`	simulators	Override the target URL (default http://localhost:8080)

A.2 Request bodies

Service	Endpoint	Body / response
target-app	`POST /api/auth/login`	`{"username","password"} -> {"accessToken","tokenType","expiresAt","username","roles"}`
target-app	`GET /api/users/me, GET /api/users`	`profile / all users (admin)`
target-app	`GET|POST /api/orders, GET /api/orders/{id}`	`{"item","quantity","amount"}`
target-app	`GET /api/admin/stats, POST /api/admin/maintenance`	`admin only`
tracker	`POST /api/incidents`	`{"title","description","severity","ruleId","fingerprint","affectedUser","sourceIp","evidence","detectedAt"}`
tracker	`PATCH /api/incidents/{id}/status`	`{"status":"RESOLVED","resolution":"...","comment":"..."}`
tracker	`PATCH /api/incidents/{id}/assign`	`{"assignee":"analyst1"}`
tracker	`POST /api/incidents/{id}/notes`	`{"content":"..."}`
detection	`POST /api/rules/run`	`-> {"fired": n, "alerts": [...]}`

A.3 Quick reference

Item	Value
Dashboard	http://localhost:3000 (analyst1 / lead)
Target API	http://localhost:8080
OpenSearch	http://127.0.0.1:9200
OpenSearch Dashboards	http://127.0.0.1:5601 (profile tools)
Seeded orders	1001, 1002 (alice); 1003, 1004 (bob)
Detection cycle	every 15 s; events searchable about 2-5 s after they happen
Incident fingerprint	RULE_ID:group-by-field:entity, e.g. BRUTE_FORCE_LOGIN:username:alice

A.4 Example: create and resolve an incident with curl

Replace 1 with the id returned by the first call

```
# via the dashboard proxy (analyst Basic auth)
curl -u analyst1:$ANALYST1_PASSWORD -H "Content-Type: application/json" \
     -d '{"title":"Phishing report from finance","severity":"MEDIUM","affectedUser":"bob"}' \
     http://localhost:3000/tracker/api/incidents
curl -u analyst1:$ANALYST1_PASSWORD -X PATCH -H "Content-Type: application/json" \
     -d '{"status":"INVESTIGATING","comment":"Contacted the user"}' \
     http://localhost:3000/tracker/api/incidents/1/status
curl -u analyst1:$ANALYST1_PASSWORD -X PATCH -H "Content-Type: application/json" \
     -d '{"status":"RESOLVED","resolution":"Credentials reset, sender blocked"}' \
     http://localhost:3000/tracker/api/incidents/1/status
```